

# 30 Años de Redes Neuronales Adaptativas: Perceptrón, Madaline y Retropropagación

BERNARD WIDROW, FELLOW, IEEE, Y MICHAEL A. LEHR

Se revisan los desarrollos fundamentales en redes neuronales artificiales de prealimentación de los últimos treinta años. El tema central de este artículo es una descripción de la historia, origen, características operativas y teoría básica de varios algoritmos de entrenamiento de redes neuronales supervisadas, incluyendo la regla del Perceptrón, el algoritmo LMS, tres reglas de Madaline y la técnica de retropropagación. Estos métodos fueron desarrollados de manera independiente, pero desde la perspectiva histórica pueden relacionarse entre sí. El concepto subyacente a estos algoritmos es el "principio de perturbación mínima", el cual sugiere que durante el entrenamiento es recomendable inyectar nueva información en una red de manera que perturbe la información almacenada en la menor medida posible.

Widrow concibió un algoritmo de aprendizaje por refuerzo denominado "caso de recompensa" o "bootstrapping" [10], [11] a mediados de la década de 1960. Este puede utilizarse para resolver problemas cuando la incertidumbre acerca de la señal de error hace que los métodos de entrenamiento supervisado resulten impracticables. Un enfoque de aprendizaje por refuerzo relacionado fue explorado posteriormente en un artículo clásico de Barto, Sutton y Anderson sobre el problema de "asignación de crédito" [12]. La técnica de Barto et al. también guarda cierto parecido con el CMAC adaptativo de Albus, un sistema distribuido de consulta en tabla basado en modelos de la memoria humana [13], [14].

Aquí está la traducción al español del texto proporcionado, siguiendo estrictamente todas las reglas especificadas en el documento.

Este año se conmemora el 30.º aniversario de la regla del Perceptrón y del algoritmo LMS, dos de las primeras reglas para el entrenamiento de elementos adaptativos. Ambos algoritmos fueron publicados por primera vez en 1960. En los años posteriores a estos descubrimientos, se han desarrollado muchas técnicas nuevas en el campo de las redes neuronales, y la disciplina está creciendo rápidamente. Un desarrollo temprano fue la Matriz de Aprendizaje de Steinhilber [1], una máquina de reconocimiento de patrones basada en funciones discriminantes lineales. Al mismo tiempo, Widrow y sus estudiantes concibieron la Regla Madaline (MRI), la primera regla de aprendizaje popular para redes neuronales con múltiples elementos adaptativos [2]. Otros trabajos tempranos incluyeron la técnica de "búsqueda de modos" de Stark, Okajima y Whipple [3]. Este fue probablemente el primer ejemplo de aprendizaje competitivo en la literatura, aunque podría argumentarse que trabajos anteriores de Rosenblatt sobre "aprendizaje espontáneo" [4], [5] merecen esta distinción. Investigaciones pioneras adicionales sobre aprendizaje competitivo y autoorganización fueron realizadas en la década de 1970 por von der Malsburg [6] y Grossberg [7]. Fukushima exploró ideas relacionadas con sus modelos Cognitron y Neocognitron de inspiración biológica [8], [9].

En la década de 1970, Grossberg desarrolló su Teoría de Resonancia Adaptativa (ART), un conjunto de hipótesis novedosas sobre los principios subyacentes que rigen los sistemas neuronales biológicos [15]. Estas ideas sirvieron como base para trabajos posteriores de Carpenter y Grossberg que involucran tres clases de arquitecturas ART: ART 1 [16], ART 2 [17] y ART 3 [18]. Estas son implementaciones neuronales autoorganizadas de algoritmos de agrupamiento de patrones. Otra teoría importante sobre sistemas autoorganizados fue pionera de Kohonen con su trabajo sobre mapas de características [19] [20].

A principios de la década de 1980, Hopfield y otros introdujeron reglas de producto externo, así como enfoques equivalentes basados en el trabajo temprano de Hebb [21] para entrenar una clase de redes recurrentes (con retroalimentación de señal) ahora denominadas modelos de Hopfield [22], [23]. Más recientemente, Kosko extendió algunas de las ideas de Hopfield y Grossberg para desarrollar su Memoria Asociativa Bidireccional (BAM) adaptativa [24], un modelo de red que emplea leyes de aprendizaje diferencial, así como hebbianas y competitivas. Otros modelos significativos de la década pasada incluyen modelos probabilísticos como la Máquina de Boltzmann de Hinton, Sejnowski y Ackley [25], [26] la cual, para simplificar en exceso, es un modelo de Hopfield que converge a soluciones mediante un proceso de recocido simulado gobernado por estadísticas de Boltzmann. La Máquina de Boltzmann se entrena mediante una ingeniosa técnica bifásica basada en Hebb.

Mientras ocurrían estos desarrollos, la investigación en sistemas adaptativos en Stanford siguió un camino independiente. Tras idear su regla Madaline, Widrow y sus estudiantes desarrollaron aplicaciones para el Adaline y el Madaline. Las primeras aplicaciones incluyeron, entre otras, el reconocimiento de voz y patrones [27], la predicción meteorológica [28] y los controles adaptativos [29]. Posteriormente, el trabajo se orientó hacia el filtrado adaptativo y el procesamiento adaptativo de señales [30], después de que los intentos por desarrollar reglas de aprendizaje para redes con múltiples capas adaptativas resultaran infructuosos. El procesamiento adaptativo de señales demostró ser

Manuscrito recibido el 12 de septiembre de 1989; revisado el 13 de abril de 1990. Este trabajo fue patrocinado por la Oficina de Ciencia y Tecnología Innovadora de la SDIO y gestionado por la ONR bajo el contrato n.º N00014-86-K-0718, por el Centro de Investigación, Desarrollo e Ingeniería de Belvoir del Departamento del Ejército bajo los contratos n.º DAAK 70-87-P-3134 y n.º DAAK 70-89-K-0001, por una subvención de Lockheed Missiles and Space Co., por la NASA bajo el contrato n.º NCA2-389, y por el Centro de Desarrollo Aéreo de Roma bajo el contrato n.º F30602-88-D-0025, subcontrato n.º E-21-T22-S1.

Los autores pertenecen al Laboratorio de Sistemas de Información, Departamento de Ingeniería Eléctrica, Universidad de Stanford, Stanford, CA 94305-4055, EE. UU.

IEEE Log Number 9038824.

0018-9219/90/0900-1415\$01.00 © 1990 IEEE

ha sido una vía fructífera para la investigación con aplicaciones que involucran antenas adaptativas [31], controles inversos adaptativos [32], cancelación adaptativa de ruido [33] y procesamiento de señales sísmicas [30]. El trabajo sobresaliente de Lucky y otros en los Laboratorios Bell condujo a importantes aplicaciones comerciales de filtros adaptativos y del algoritmo LMS para la ecualización adaptativa en módems de alta velocidad [34], [35] y para canceladores de eco adaptativos en circuitos telefónicos de larga distancia y satelitales [36]. Después de 20 años de investigación en procesamiento adaptativo de señales, el trabajo en el laboratorio de Widrow ha retornado una vez más a las redes neuronales.

La primera extensión importante de la red neuronal prealimentada más allá de Madaline ocurrió en 1971, cuando Werbos desarrolló un algoritmo de entrenamiento de retropropagación que, en 1974, publicó por primera vez en su tesis doctoral [37]. Desafortunadamente, el trabajo de Werbos permaneció casi desconocido en la comunidad científica. En 1982, Parker redescubrió la técnica [39] y, en 1985, publicó un informe sobre ella en el M.I.T. [40]. Poco después de que Parker publicara sus hallazgos, Rumelhart, Hinton y Williams [41], [42] también redescubrieron la técnica y, en gran medida como resultado del marco claro dentro del cual presentaron sus ideas, finalmente lograron dar la a conocer ampliamente.

Los elementos utilizados por Rumelhart et al. en la red de retropropagación difieren de aquellos empleados en las arquitecturas Madaline anteriores. Los elementos adaptativos en la estructura Madaline original utilizaban cuantificadores de limitación dura (signos), mientras que los elementos en la red de retropropagación emplean únicamente no linealidades diferenciables, o funciones "sigmoide". En implementaciones digitales, el cuantificador de limitación dura se calcula más fácilmente que cualquiera de las no linealidades diferenciables utilizadas en las redes de retropropagación. En 1987, Widrow, Winter y Baxter revisaron el algoritmo Madaline original con el objetivo de desarrollar una nueva técnica que pudiera adaptar múltiples capas de elementos adaptativos utilizando los cuantificadores de limitación dura más simples. El resultado fue la Regla Madaline #1 [43].

David Andes, del Centro de Armas Navales de los Estados Unidos en China Lake, CA, modificó el Madaline II en 1988 al reemplazar los cuantificadores de limitación abrupta en las funciones Adaline y sigmoide, inventando así la Regla Madaline II (MR-II). Widrow y sus estudiantes fueron los primeros en reconocer que esta regla es matemáticamente equivalente a la retropropagación.

El esquema anterior ofrece solo una visión parcial de la disciplina, y muchos de los cubrimientos emblemáticos no han sido mencionados. No hace falta decir que el campo de las redes neuronales se está convirtiendo rápidamente en uno vasto, y en una breve reseña no podríamos aspirar a cubrir todo el tema con detalle. En consecuencia, muchos desarrollos significativos, incluyendo algunos de los mencionados anteriormente, no se discuten en este artículo. Los algoritmos descritos se limitan principalmente a

Debemos señalar, sin embargo, que en el campo del cálculo variacional la idea de la retropropagación del error a través de sistemas no lineales existió siglos antes de que Werbos pensara por primera vez en aplicar este concepto a las redes neuronales. En los últimos 25 años, estos métodos se han utilizado ampliamente en el campo del control óptimo, según lo discutido por Le Cun [38].

El término-sigmoide se utiliza generalmente en referencia a funciones monótonamente crecientes con forma de S, como la tangente hiperbólica. Sin embargo, en este artículo empleamos dicho término para designar cualquier función no lineal suave en la salida de un elemento adaptativo lineal. En otros trabajos, estas no linealidades reciben diversos nombres, como funciones de compresión, funciones de activación, características de transferencia o funciones de umbral.

Información sobre los paradigmas de redes neuronales no discutidos en este artículo puede obtenerse de diversas fuentes, como el estudio conciso de Lippmann [44] y la colección de clásicos de Anderson y Rosenfeld [45]. Gran parte del trabajo temprano en el campo, de la década de 1960, es revisado cuidadosamente en la monografía de Nilsson [46]. Una buena perspectiva de algunos de los resultados más recientes se presenta en el popular conjunto de tres volúmenes de Rumelhart y McClelland [47]. Un artículo de Moore [48] ofrece una discusión clara sobre ART 1 y parte de la terminología de Grossberg. Otro recurso es el informe del estudio DARPA [49], que proporciona una "instantánea" muy completa y legible del campo en 1988.

A continuación se presenta la traducción del texto proporcionado al español académico formal, siguiendo estrictamente el original.

Hoy en día podemos construir computadoras y otras máquinas que realizan una variedad de tareas bien definidas con una rapidez y fiabilidad inigualables por los humanos. Ningún ser humano puede invertir matrices o resolver sistemas de ecuaciones diferenciales a velocidades que rivalicen con las estaciones de trabajo modernas. No obstante, muchos problemas que ninguna máquina creada por el hombre ha logrado resolver de manera satisfactoria son fácilmente desentrañados por las capacidades perceptivas o cognitivas de los humanos, y a menudo de mamíferos inferiores, o incluso de peces e insectos. Ningún sistema de visión artificial puede igualar la capacidad humana para reconocer imágenes visuales formadas por objetos de todas las formas y orientaciones bajo una amplia gama de condiciones. Los humanos reconocen sin esfuerzo objetos en entornos diversos y condiciones de iluminación variables, incluso cuando están oscurecidos por suciedad u ocluidos por otros objetos. Asimismo, el rendimiento de la tecnología actual de reconocimiento de voz palidece en comparación con el desempeño de un adulto humano, que reconoce fácilmente palabras pronunciadas por diferentes personas, a diferentes velocidades, tonos y volúmenes, incluso en presencia de distorsión o ruido de fondo.

Los problemas que el cerebro resuelve de manera más efectiva que la computadora digital suelen presentar dos características: generalmente están mal definidos y, por lo general, requieren una enorme cantidad de procesamiento. Reconocer el carácter de un objeto a partir de su imagen en televisión, por ejemplo, implica resolver ambigüedades asociadas con la distorsión y la iluminación. También implica completar información sobre una escena tridimensional que falta en la imagen bidimensional de la pantalla. Un número infinito de escenas tridimensionales puede proyectarse en una imagen bidimensional. No obstante, el cerebro maneja bien esta ambigüedad y, utilizando señales aprendidas, generalmente tiene poca dificultad para determinar correctamente el papel que desempeña la dimensión faltante.

Como cualquier persona que haya realizado incluso operaciones de filtrado simples sobre imágenes sabe, el procesamiento de imágenes de alta resolución requiere una gran cantidad de cómputo. Nuestros cerebros logran esto mediante la utilización de un paralelismo masivo, con millones e incluso miles de millones de neuronas en partes del cerebro trabajando juntas para resolver problemas complejos. Debido a que los amplificadores operacionales de estado sólido y las compuertas lógicas pueden computar

muchos órdenes de magnitud más rápida que las estimaciones actuales de la velocidad computacional de las neuronas en el cerebro, pronto podremos construir máquinas relativamente económicas con la capacidad de procesar tanta información como el cerebro humano. Sin embargo, este enorme poder de procesamiento servirá de poco para ayudarnos a resolver problemas, a menos que podamos utilizarlo de manera efectiva. Por ejemplo, coordinar muchos miles de procesadores, que deben cooperar eficientemente para resolver un problema, no es una tarea sencilla. Si cada procesador debe programarse por separado, y si todas las contingencias asociadas con diversas ambigüedades deben diseñarse en el software, incluso un problema relativamente simple puede volverse rápidamente inmanejable. El lento progreso durante los últimos 25 años aproximadamente en la visión artificial y otras áreas de la inteligencia artificial es un testimonio de las dificultades asociadas con la resolución de problemas ambiguos e intensivos en computación en computadoras von Neumann y arquitecturas relacionadas.

Output

Weight Vector

Desired Response

Fig. 1. Combinador lineal adaptativo.

valores analógicos continuos o valores binarios. Los pesos son esencialmente continuamente variables y pueden adoptar valores tanto negativos como positivos.

Durante el proceso de entrenamiento, los patrones de entrada y las respuestas deseadas correspondientes se presentan al combinador lineal. Un algoritmo de adaptación ajusta automáticamente los pesos de modo que las respuestas de salida a los patrones de entrada sean lo más cercanas posible a sus respectivas respuestas deseadas. En aplicaciones de procesamiento de señales, el método más popular para adaptar los pesos es el algoritmo LMS (mínimos cuadrados medios) simple [58], [59], a menudo denominado regla delta de Widrow-Hoff [42]. Este algoritmo minimiza la suma de los cuadrados de los errores lineales sobre el conjunto de entrenamiento. El error lineal  $(e_k)$  se define como la diferencia entre la respuesta deseada  $(d_k)$  y la salida lineal  $(s_k)$  durante la presentación  $(k)$ . Disponer de esta señal de error es necesario para adaptar los pesos. Sin embargo, cuando el combinador lineal adaptativo está integrado en una red neuronal multielemento, a menudo no se dispone directamente de una señal de error para cada combinador lineal individual, y deben idearse procedimientos más complejos para adaptar los vectores de pesos. Estos procedimientos constituyen el enfoque principal de este artículo.

Por lo tanto, existe cierta razón para considerar abordar ciertos problemas mediante el diseño de computadoras naturalmente paralelas, que procesan información y toman decisiones de manera paralela.

Hoy en día, la mayor parte de la investigación y aplicación de las redes neuronales artificiales se realiza mediante la simulación de redes en computadoras seriadas. Las limitaciones de velocidad mantienen dichas redes relativamente pequeñas, pero incluso con redes pequeñas se han abordado algunos problemas sorprendentemente difíciles. Redes con menos de 150 elementos neuronales se han utilizado con éxito en simulaciones de control vehicular [50], generación de voz [51], [52] y detección de minas submarinas [49]. También se han empleado redes pequeñas con éxito en la detección de explosivos en aeropuertos [53], sistemas expertos [54], [55] y decenas de otras aplicaciones. Además, los esfuerzos por desarrollar hardware de redes neuronales en paralelo están logrando cierto éxito, y dicho hardware debería estar disponible en el futuro para abordar problemas más difíciles, como el reconocimiento de voz [56], [57].

#### Un Clasificador Lineal: El Elemento de Umbral Simple

El bloque fundamental utilizado en muchas redes neuronales es el "elemento lineal adaptativo", o Adaline [58] (Fig. 2).

Este es un elemento de lógica de umbral adaptativo. Consiste en un combinador lineal adaptativo en cascada con un cuantificador de limitación dura, el cual se utiliza para producir una salida binaria,  $Y = \text{sgn}(s)$ . El peso de sesgo  $w_0$ , que está conectado a una entrada constante  $x_0 = 1$ , controla efectivamente el nivel de umbral del cuantificador.

Ya sea implementada en hardware paralelo o simulada en una computadora, toda red neuronal consiste en una colección de elementos simples que trabajan en conjunto para resolver problemas. Un componente fundamental de casi todas las redes neuronales artificiales, y de la mayoría de los demás sistemas adaptativos, es el combinador lineal adaptativo.

En las redes neuronales de un solo elemento, a menudo se emplea un algoritmo adaptativo (como el algoritmo LMS o la regla del perceptrón) para ajustar los pesos del Adaline, de modo que responda correctamente al mayor número posible de patrones en un conjunto de entrenamiento que posee respuestas deseadas binarias. Una vez ajustados los pesos, las respuestas del elemento entrenado o pueden evaluarse aplicando diversos patrones de entrada. Si el Adaline responde con alta probabilidad de forma correcta a patrones de entrada que no estaban incluidos en el conjunto de entrenamiento, se dice que se ha producido una generalización. El aprendizaje y la generalización se encuentran entre los atributos más útiles de los Adalines y las redes neuronales.

#### A. El Combinador Lineal Adaptativo

El combinador lineal adaptativo se diagrama en la Fig. 1. Su salida es una combinación lineal de sus entradas. En una implementación digital, este elemento recibe en el instante  $k$  un vector de señal de entrada o vector de patrón de entrada  $x(k) = [x_0(k), x_1(k), x_2(k), \dots, x_n(k)]^T$  y una respuesta deseada  $d(k)$ , una entrada especial utilizada para efectuar el aprendizaje. Los componentes del vector de entrada se ponderan mediante un conjunto de coeficientes, el vector de pesos  $w(k) = [w_0(k), w_1(k), w_2(k), \dots, w_n(k)]^T$ . Luego se calcula la suma de las entradas ponderadas, produciendo una salida lineal, el producto interno  $s(k) = x(k)^T w(k)$ . Los componentes de  $x(k)$  pueden ser

Separabilidad Lineal: Con  $n$  entradas binarias y una salida binaria

En la literatura de redes neuronales, estos elementos suelen denominarse "neuroas adaptativas". Sin embargo, en una conversación entre David Hubel de la Escuela de Medicina de Harvard y Bernard Widrow, el Dr. Hubel señaló que la Adaline difiere de la neurona biológica en que contiene no solo el cuerpo de la célula neuronal, sino también las sinapsis de entrada y un mecanismo para entrenarlas.

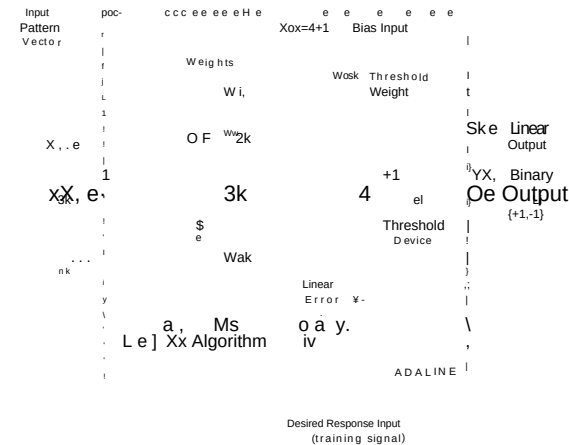


Fig. 2. Elemento lineal adaptativo (Adaline).

A la salida, un único Adaline del tipo mostrado en la Fig. 2 es capaz de implementar ciertas funciones lógicas. Existen  $2^n$  patrones de entrada posibles. Una implementación lógica general sería capaz de clasificar cada patrón como  $\{+1\}$  o  $\{-1\}$ , de acuerdo con la respuesta deseada. Por lo tanto, existen  $2^{2^n}$  funciones lógicas posibles que conectan  $n$  entradas con una única salida binaria. Un único Adaline solo puede realizar un subconjunto reducido de estas funciones, conocidas como funciones lógicas linealmente separables o funciones de lógica de umbral [60]. Estas constituyen el conjunto de funciones lógicas que pueden obtenerse con todas las variaciones posibles de pesos.

La Figura 3 muestra un elemento Adaline de dos entradas. La Figura 4 representa a todas las entradas binarias posibles para este elemento mediante cuatro puntos grades en el espacio del vector de patrones. En este espacio, los componentes del vector de patrón de entrada se sitúan a lo largo de los ejes de coordenadas. El Adaline separa los patrones de entrada en dos categorías, dependiendo de los valores de los pesos. Un aspecto crítico

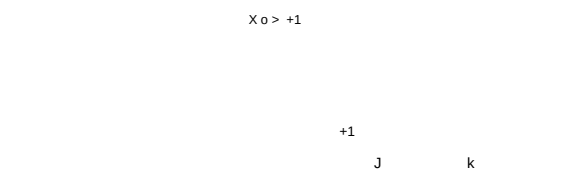


Fig. 3. Adaline de dos entradas.

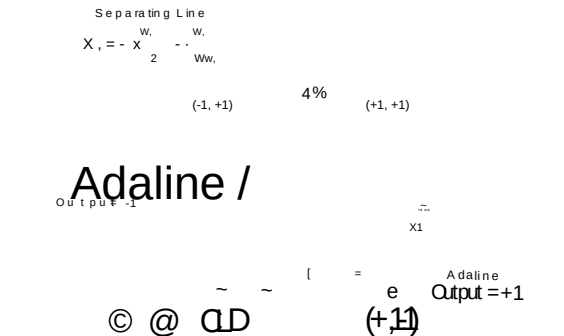


Fig. 4. Línea de separación en el espacio de patrones.

La condición de umbralización ocurre cuando la salida lineal  $(s)$  es igual a cero:  $[s = 0]$

$$S = X_1W_1 + X_2W_2 + W_0 = 0, \quad (1)$$

Por lo tanto

$$X = x_{w_2} \quad y \quad \frac{w}{w_2} \quad OF \quad (2)$$

La Figura 4 representa gráficamente esta relación lineal, la cual comprende una línea de separación cuya pendiente e intersección están dadas por

$$\text{slope} = \frac{w_1}{w_2}$$

$$\text{intercept} = -2. \quad \frac{w}{w_2} \quad (3)$$

Los tres pesos determinan la pendiente, la intersección y el lado de la línea de separación que corresponde a una salida positiva. El lado opuesto de la línea de separación corresponde a una salida negativa. Para Adalines con cuatro pesos, la frontera de separación es un plano; con más de cuatro pesos, la frontera es un hiperplano. Nótese que si el peso de sesgo es cero, el hiperplano de separación será homocéntrico a través del origen en el espacio de patrones.

Tal como se esboza en la Fig. 4, los patrones de entrada binarios se clasifican de la siguiente manera:

$$\begin{aligned} (+1, +1) &> +1 \\ (+1, -1) &> +1 \\ (-1, -1) &> +1 \\ (-1, +1) &> -1 \end{aligned} \quad (4)$$

Este es un ejemplo de una función linealmente separable. Un ejemplo de una función que no es linealmente separable es la función O exclusiva (XOR) de dos entradas:

$$\begin{aligned} (+1, +1) &> +1 \\ (+1, -1) &> -1 \\ (-1, -1) &> +1 \\ (-1, +1) &> -1 \end{aligned} \quad (5)$$

No existe una única línea recta que pueda lograr esta separación de los patrones de entrada; por lo tanto, sin preprocesamiento, ningún Adaline individual puede implementar la función O exclusiva (XOR).

Con dos entradas, un único Adaline puede realizar 14 de las 16 funciones lógicas posibles. Sin embargo, con muchas entradas, solo una pequeña fracción de todas las funciones lógicas posibles es realizable, es decir, aquellas que son linealmente separables. Se pueden utilizar combinaciones de elementos o redes de elementos para realizar funciones que no son linealmente separables.

**\*\*Capacidad de los Clasificadores Lineales:\*\*** El número de patrones de entrenamiento o estímulos que un Adaline puede aprender a clasificar correctamente es un aspecto importante. Cada combinación de patrón y salida deseada representa una restricción de desigualdad sobre los pesos. Es posible que existan inconsistencias en conjuntos de desigualdades simultáneas, al igual que ocurre con las igualdades simultáneas. Cuando las desigualdades (es decir, los patrones) se determinan de forma aleatoria, el número de estas que se puede seleccionar antes de que surja una inconsistencia es una cuestión de probabilidad.

En sus disertaciones de 1964 [61], [62], T. M. Cover y R. J. Brown demostraron que el número promedio de patrones aleatorios con respuestas deseadas binarias aleatorias que pueden ser

absorbido por un Adaline es aproximadamente igual al doble del número de pesos de valor continuo y están distribuidos suavemente (es decir, las posiciones de los patrones se generan mediante una función de distribución que no contiene impulsos), se garantiza que dicho conjunto de entrenamiento pueda ser separado por un Adaline (es decir, los resultados de capacidad representan cotas superiores útiles). La probabilidad es una función de  $(N_t)$ , el número de patrones de entrada en el conjunto de entrenamiento, y  $(N_w)$ , el número de pesos en el Adaline, incluyendo el peso de sesgo, si se utiliza:

$$P_{\text{Separable}} = \frac{1}{2} \sum_{i=0}^{N_w-1} \left( \frac{N_t}{N_w} \right)^i \quad \text{for } N_t > N_w \quad (6)$$

En la Fig. 5, esta fórmula se utilizó para trazar un conjunto de curvas analíticas, las cuales muestran la probabilidad de que un conjunto de  $N$  patrones aleatorios pueda ser entrenado en un Adaline en función de la relación  $N_t/N_w$ . Nótese en estas curvas que, a medida que aumenta el número de pesos, la capacidad estadística de patrones del Adaline  $C_s = 2N_w$ , se convierte en una estimación precisa del número de respuestas que puede aprender.

Otro hecho que puede observarse en la Fig. 5 es que un

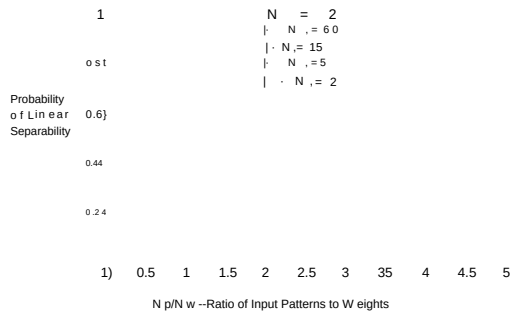


Fig. 5. Probabilidad de que un Adaline pueda separar un conjunto de patrones de entrenamiento en función de la relación  $N_t/N_w$ .

**\*\*Problema\*\*** se garantiza que tiene solución si el número de patrones es igual (o menor) a la mitad de la capacidad estadística de patrones; es decir, si el número de patrones es igual al número de pesos. Nos referiremos a esto como la capacidad determinista de patrones  $(C_d)$  del Adaline. Un Adaline puede aprender cualquier tarea de clasificación de patrones de dos categorías que involucre no más patrones que los representados por su capacidad determinista,  $(C_d(N_t))$ .

Los resultados de capacidad se aplican a patrones de entrenamiento seleccionados aleatoriamente. En la mayoría de los problemas de interés, los patrones en el conjunto de entrenamiento no son aleatorios, sino que presentan ciertas regularidades estadísticas. Estas regularidades son lo que hace posible la generalización. El número de patrones que un Adaline puede aprender en un problema práctico a menudo supera a con creces su capacidad estadística, ya que el Adaline es capaz de generalizar dentro del conjunto de entrenamiento y aprende muchos de los patrones de entrenamiento incluso antes de que estos sean presentados.

Aquí está la traducción al español académico formal, siguiendo todas las reglas específicas de los clasificadores No Lineales:

El clasificador lineal tiene una capacidad limitada y, por supuesto, se restringe únicamente a formas de discriminación de patrones linealmente separables. Los clasificadores más sofisticados con capacidades superiores son no lineales. En este documento se describen dos tipos de clasificadores no lineales. El primero consiste en una red de preprocesamiento fija conectada a un único elemento adaptativo, y el otro es la red neuronal de prealimentación multicapa.

**\*\*Funciones Discriminantes Polinomiales:\*\*** La aplicación de funciones no lineales de las entradas a un único Adaline puede generar fronteras de decisión no lineales. Entre las no linealidades útiles se incluyen las funciones polinomiales. Considere el sistema ilustrado en la Fig. 6, el cual contiene únicamente entradas lineales y cuadráticas.

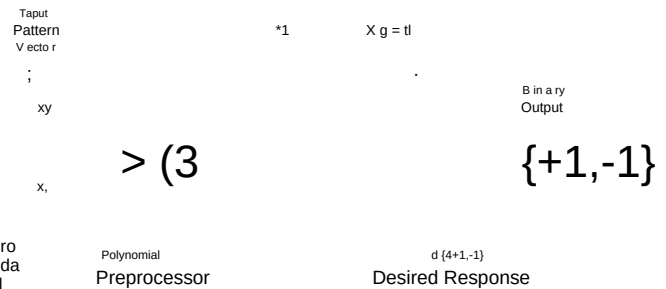


Fig. 6. Adaline con entradas mapeadas a través de no linealidades.

funciones. La condición crítica de umbral para este sistema es

Ambos resultados de capacidad, tanto el estadístico como el determinista, dependen de una condición leve sobre las posiciones de los patrones de entrada: los patrones deben estar en posición general con respecto al Adaline. Si los patrones de entrada a un Adaline

$$S = w_0 + x_1w_1 + x_2w_2 + x_3w_3 + x_4w_4 + x_5w_5 + x_6w_6 + x_7w_7 + x_8w_8 + x_9w_9 + x_{10}w_{10} + x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + x_{14}w_{14} + x_{15}w_{15} + x_{16}w_{16} + x_{17}w_{17} + x_{18}w_{18} + x_{19}w_{19} + x_{20}w_{20} + x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + x_{24}w_{24} + x_{25}w_{25} + x_{26}w_{26} + x_{27}w_{27} + x_{28}w_{28} + x_{29}w_{29} + x_{30}w_{30} + x_{31}w_{31} + x_{32}w_{32} + x_{33}w_{33} + x_{34}w_{34} + x_{35}w_{35} + x_{36}w_{36} + x_{37}w_{37} + x_{38}w_{38} + x_{39}w_{39} + x_{40}w_{40} + x_{41}w_{41} + x_{42}w_{42} + x_{43}w_{43} + x_{44}w_{44} + x_{45}w_{45} + x_{46}w_{46} + x_{47}w_{47} + x_{48}w_{48} + x_{49}w_{49} + x_{50}w_{50} + x_{51}w_{51} + x_{52}w_{52} + x_{53}w_{53} + x_{54}w_{54} + x_{55}w_{55} + x_{56}w_{56} + x_{57}w_{57} + x_{58}w_{58} + x_{59}w_{59} + x_{60}w_{60} + x_{61}w_{61} + x_{62}w_{62} + x_{63}w_{63} + x_{64}w_{64} + x_{65}w_{65} + x_{66}w_{66} + x_{67}w_{67} + x_{68}w_{68} + x_{69}w_{69} + x_{70}w_{70} + x_{71}w_{71} + x_{72}w_{72} + x_{73}w_{73} + x_{74}w_{74} + x_{75}w_{75} + x_{76}w_{76} + x_{77}w_{77} + x_{78}w_{78} + x_{79}w_{79} + x_{80}w_{80} + x_{81}w_{81} + x_{82}w_{82} + x_{83}w_{83} + x_{84}w_{84} + x_{85}w_{85} + x_{86}w_{86} + x_{87}w_{87} + x_{88}w_{88} + x_{89}w_{89} + x_{90}w_{90} + x_{91}w_{91} + x_{92}w_{92} + x_{93}w_{93} + x_{94}w_{94} + x_{95}w_{95} + x_{96}w_{96} + x_{97}w_{97} + x_{98}w_{98} + x_{99}w_{99} + x_{100}w_{100} + x_{101}w_{101} + x_{102}w_{102} + x_{103}w_{103} + x_{104}w_{104} + x_{105}w_{105} + x_{106}w_{106} + x_{107}w_{107} + x_{108}w_{108} + x_{109}w_{109} + x_{110}w_{110} + x_{111}w_{111} + x_{112}w_{112} + x_{113}w_{113} + x_{114}w_{114} + x_{115}w_{115} + x_{116}w_{116} + x_{117}w_{117} + x_{118}w_{118} + x_{119}w_{119} + x_{120}w_{120} + x_{121}w_{121} + x_{122}w_{122} + x_{123}w_{123} + x_{124}w_{124} + x_{125}w_{125} + x_{126}w_{126} + x_{127}w_{127} + x_{128}w_{128} + x_{129}w_{129} + x_{130}w_{130} + x_{131}w_{131} + x_{132}w_{132} + x_{133}w_{133} + x_{134}w_{134} + x_{135}w_{135} + x_{136}w_{136} + x_{137}w_{137} + x_{138}w_{138} + x_{139}w_{139} + x_{140}w_{140} + x_{141}w_{141} + x_{142}w_{142} + x_{143}w_{143} + x_{144}w_{144} + x_{145}w_{145} + x_{146}w_{146} + x_{147}w_{147} + x_{148}w_{148} + x_{149}w_{149} + x_{150}w_{150} + x_{151}w_{151} + x_{152}w_{152} + x_{153}w_{153} + x_{154}w_{154} + x_{155}w_{155} + x_{156}w_{156} + x_{157}w_{157} + x_{158}w_{158} + x_{159}w_{159} + x_{160}w_{160} + x_{161}w_{161} + x_{162}w_{162} + x_{163}w_{163} + x_{164}w_{164} + x_{165}w_{165} + x_{166}w_{166} + x_{167}w_{167} + x_{168}w_{168} + x_{169}w_{169} + x_{170}w_{170} + x_{171}w_{171} + x_{172}w_{172} + x_{173}w_{173} + x_{174}w_{174} + x_{175}w_{175} + x_{176}w_{176} + x_{177}w_{177} + x_{178}w_{178} + x_{179}w_{179} + x_{180}w_{180} + x_{181}w_{181} + x_{182}w_{182} + x_{183}w_{183} + x_{184}w_{184} + x_{185}w_{185} + x_{186}w_{186} + x_{187}w_{187} + x_{188}w_{188} + x_{189}w_{189} + x_{190}w_{190} + x_{191}w_{191} + x_{192}w_{192} + x_{193}w_{193} + x_{194}w_{194} + x_{195}w_{195} + x_{196}w_{196} + x_{197}w_{197} + x_{198}w_{198} + x_{199}w_{199} + x_{200}w_{200} + x_{201}w_{201} + x_{202}w_{202} + x_{203}w_{203} + x_{204}w_{204} + x_{205}w_{205} + x_{206}w_{206} + x_{207}w_{207} + x_{208}w_{208} + x_{209}w_{209} + x_{210}w_{210} + x_{211}w_{211} + x_{212}w_{212} + x_{213}w_{213} + x_{214}w_{214} + x_{215}w_{215} + x_{216}w_{216} + x_{217}w_{217} + x_{218}w_{218} + x_{219}w_{219} + x_{220}w_{220} + x_{221}w_{221} + x_{222}w_{222} + x_{223}w_{223} + x_{224}w_{224} + x_{225}w_{225} + x_{226}w_{226} + x_{227}w_{227} + x_{228}w_{228} + x_{229}w_{229} + x_{230}w_{230} + x_{231}w_{231} + x_{232}w_{232} + x_{233}w_{233} + x_{234}w_{234} + x_{235}w_{235} + x_{236}w_{236} + x_{237}w_{237} + x_{238}w_{238} + x_{239}w_{239} + x_{240}w_{240} + x_{241}w_{241} + x_{242}w_{242} + x_{243}w_{243} + x_{244}w_{244} + x_{245}w_{245} + x_{246}w_{246} + x_{247}w_{247} + x_{248}w_{248} + x_{249}w_{249} + x_{250}w_{250} + x_{251}w_{251} + x_{252}w_{252} + x_{253}w_{253} + x_{254}w_{254} + x_{255}w_{255} + x_{256}w_{256} + x_{257}w_{257} + x_{258}w_{258} + x_{259}w_{259} + x_{260}w_{260} + x_{261}w_{261} + x_{262}w_{262} + x_{263}w_{263} + x_{264}w_{264} + x_{265}w_{265} + x_{266}w_{266} + x_{267}w_{267} + x_{268}w_{268} + x_{269}w_{269} + x_{270}w_{270} + x_{271}w_{271} + x_{272}w_{272} + x_{273}w_{273} + x_{274}w_{274} + x_{275}w_{275} + x_{276}w_{276} + x_{277}w_{277} + x_{278}w_{278} + x_{279}w_{279} + x_{280}w_{280} + x_{281}w_{281} + x_{282}w_{282} + x_{283}w_{283} + x_{284}w_{284} + x_{285}w_{285} + x_{286}w_{286} + x_{287}w_{287} + x_{288}w_{288} + x_{289}w_{289} + x_{290}w_{290} + x_{291}w_{291} + x_{292}w_{292} + x_{293}w_{293} + x_{294}w_{294} + x_{295}w_{295} + x_{296}w_{296} + x_{297}w_{297} + x_{298}w_{298} + x_{299}w_{299} + x_{300}w_{300} + x_{301}w_{301} + x_{302}w_{302} + x_{303}w_{303} + x_{304}w_{304} + x_{305}w_{305} + x_{306}w_{306} + x_{307}w_{307} + x_{308}w_{308} + x_{309}w_{309} + x_{310}w_{310} + x_{311}w_{311} + x_{312}w_{312} + x_{313}w_{313} + x_{314}w_{314} + x_{315}w_{315} + x_{316}w_{316} + x_{317}w_{317} + x_{318}w_{318} + x_{319}w_{319} + x_{320}w_{320} + x_{321}w_{321} + x_{322}w_{322} + x_{323}w_{323} + x_{324}w_{324} + x_{325}w_{325} + x_{326}w_{326} + x_{327}w_{327} + x_{328}w_{328} + x_{329}w_{329} + x_{330}w_{330} + x_{331}w_{331} + x_{332}w_{332} + x_{333}w_{333} + x_{334}w_{334} + x_{335}w_{335} + x_{336}w_{336} + x_{337}w_{337} + x_{338}w_{338} + x_{339}w_{339} + x_{340}w_{340} + x_{341}w_{341} + x_{342}w_{342} + x_{343}w_{343} + x_{344}w_{344} + x_{345}w_{345} + x_{346}w_{346} + x_{347}w_{347} + x_{348}w_{348} + x_{349}w_{349} + x_{350}w_{350} + x_{351}w_{351} + x_{352}w_{352} + x_{353}w_{353} + x_{354}w_{354} + x_{355}w_{355} + x_{356}w_{356} + x_{357}w_{357} + x_{358}w_{358} + x_{359}w_{359} + x_{360}w_{360} + x_{361}w_{361} + x_{362}w_{362} + x_{363}w_{363} + x_{364}w_{364} + x_{365}w_{365} + x_{366}w_{366} + x_{367}w_{367} + x_{368}w_{368} + x_{369}w_{369} + x_{370}w_{370} + x_{371}w_{371} + x_{372}w_{372} + x_{373}w_{373} + x_{374}w_{374} + x_{375}w_{375} + x_{376}w_{376} + x_{377}w_{377} + x_{378}w_{378} + x_{379}w_{379} + x_{380}w_{380} + x_{381}w_{381} + x_{382}w_{382} + x_{383}w_{383} + x_{384}w_{384} + x_{385}w_{385} + x_{386}w_{386} + x_{387}w_{387} + x_{388}w_{388} + x_{389}w_{389} + x_{390}w_{390} + x_{391}w_{391} + x_{392}w_{392} + x_{393}w_{393} + x_{394}w_{394} + x_{395}w_{395} + x_{396}w_{396} + x_{397}w_{397} + x_{398}w_{398} + x_{399}w_{399} + x_{400}w_{400} + x_{401}w_{401} + x_{402}w_{402} + x_{403}w_{403} + x_{404}w_{404} + x_{405}w_{405} + x_{406}w_{406} + x_{407}w_{407} + x_{408}w_{408} + x_{409}w_{409} + x_{410}w_{410} + x_{411}w_{411} + x_{412}w_{412} + x_{413}w_{413} + x_{414}w_{414} + x_{415}w_{415} + x_{416}w_{416} + x_{417}w_{417} + x_{418}w_{418} + x_{419}w_{419} + x_{420}w_{420} + x_{421}w_{421} + x_{422}w_{422} + x_{423}w_{423} + x_{424}w_{424} + x_{425}w_{425} + x_{426}w_{426} + x_{427}w_{427} + x_{428}w_{428} + x_{429}w_{429} + x_{430}w_{430} + x_{431}w_{431} + x_{432}w_{432} + x_{433}w_{433} + x_{434}w_{434} + x_{435}w_{435} + x_{436}w_{436} + x_{437}w_{437} + x_{438}w_{438} + x_{439}w_{439} + x_{440}w_{440} + x_{441}w_{441} + x_{442}w_{442} + x_{443}w_{443} + x_{444}w_{444} + x_{445}w_{445} + x_{446}w_{446} + x_{447}w_{447} + x_{448}w_{448} + x_{449}w_{449} + x_{450}w_{450} + x_{451}w_{451} + x_{452}w_{452} + x_{453}w_{453} + x_{454}w_{454} + x_{455}w_{455} + x_{456}w_{456} + x_{457}w_{457} + x_{458}w_{458} + x_{459}w_{459} + x_{460}w_{460} + x_{461}w_{461} + x_{462}w_{462} + x_{463}w_{463} + x_{464}w_{464} + x_{465}w_{465} + x_{466}w_{466} + x_{467}w_{467} + x_{468}w_{468} + x_{469}w_{469} + x_{470}w_{470} + x_{471}w_{471} + x_{472}w_{472} + x_{473}w_{473} + x_{474}w_{474} + x_{475}w_{475} + x_{476}w_{476} + x_{477}w_{477} + x_{478}w_{478} + x_{479}w_{479} + x_{480}w_{480} + x_{481}w_{481} + x_{482}w_{482} + x_{483}w_{483} + x_{484}w_{484} + x_{485}w_{485} + x_{486}w_{486} + x_{487}w_{487} + x_{488}w_{488} + x_{489}w_{489} + x_{490}w_{490} + x_{491}w_{491} + x_{492}w_{492} + x_{493}w_{493} + x_{494}w_{494} + x_{495}w_{495} + x_{496}w_{496} + x_{497}w_{497} + x_{498}w_{498} + x_{499}w_{499} + x_{500}w_{500} + x_{501}w_{501} + x_{502}w_{502} + x_{503}w_{503} + x_{504}w_{504} + x_{505}w_{505} + x_{506}w_{506} + x_{507}w_{507} + x_{508}w_{508} + x_{509}w_{509} + x_{510}w_{510} + x_{511}w_{511} + x_{512}w_{512} + x_{513}w_{513} + x_{514}w_{514} + x_{515}w_{515} + x_{516}w_{516} + x_{517}w_{517} + x_{518}w_{518} + x_{519}w_{519} + x_{520}w_{520} + x_{521}w_{521} + x_{522}w_{522} + x_{523}w_{523} + x_{524}w_{524} + x_{525}w_{525} + x_{526}w_{526} + x_{527}w_{527} + x_{528}w_{528} + x_{529}w_{529} + x_{530}w_{530} + x_{531}w_{531} + x_{532}w_{532} + x_{533}w_{533} + x_{534}w_{534} + x_{535}w_{535} + x_{536}w_{536} + x_{537}w_{537} + x_{538}w_{538} + x_{539}w_{539} + x_{540}w_{540} + x_{541}w_{541} + x_{542}w_{542} + x_{543}w_{543} + x_{544}w_{544} + x_{545}w_{545} + x_{546}w_{546} + x_{547}w_{547} + x_{548}w_{548} + x_{549}w_{549} + x_{550}w_{550} + x_{551}w_{551} + x_{552}w_{552} + x_{553}w_{553} + x_{554}w_{554} + x_{555}w_{555} + x_{556}w_{556} + x_{557}w_{557} + x_{558}w_{558} + x_{559}w_{559} + x_{560}w_{560} + x_{561}w_{561} + x_{562}w_{562} + x_{563}w_{563} + x_{564}w_{564} + x_{565}w_{565} + x_{566}w_{566} + x_{567}w_{567} + x_{568}w_{568} + x_{569}w_{569} + x_{570}w_{570} + x_{571}w_{571} + x_{572}w_{572} + x_{573}w_{573} + x_{574}w_{574} + x_{575}w_{575} + x_{576}w_{576} + x_{577}w_{577} + x_{578}w_{578} + x_{579}w_{579} + x_{580}w_{580} + x_{581}w_{581} + x_{582}w_{582} + x_{583}w_{583} + x_{584}w_{584} + x_{585}w_{585} + x_{586}w_{586} + x_{587}w_{587} + x_{588}w_{588} + x_{589}w_{589} + x_{590}w_{590} + x_{591}w_{591} + x_{592}w_{592} + x_{593}w_{593} + x_{594}w_{594} + x_{595}w_{595} + x_{596}w_{596} + x_{597}w_{597} + x_{598}w_{598} + x_{599}w_{599} + x_{600}w_{600} + x_{601}w_{601} + x_{602}w_{602} + x_{603}w_{603} + x_{604}w_{604} + x_{605}w_{605} + x_{606}w_{606} + x_{607}w_{607} + x_{608}w_{608} + x_{609}w_{609} + x_{610}w_{610} + x_{611}w_{611} + x_{612}w_{612} + x_{613}w_{613} + x_{614}w_{614} + x_{615}w_{615} + x_{616}w_{616} + x_{617}w_{617} + x_{618}w_{618} + x_{619}w_{619} + x_{620}w_{620} + x_{621}w_{621} + x_{622}w_{622} + x_{623}w_{623} + x_{624}w_{624} + x_{625}w_{625} + x_{626}w_{626} + x_{627}w_{627} + x_{628}w_{628} + x_{629}w_{629} + x_{630}w_{630} + x_{631}w_{631} + x_{632}w_{632} + x_{633}w_{633} + x_{634}w_{634} + x_{635}w_{635} + x_{636}w_{636} + x_{637}w_{637} + x_{638}w_{638} + x_{639}w_{639} + x_{640}w_{640} + x_{641}w_{641} + x_{642}w_{642} + x_{643}w_{643} + x_{644}w_{644} + x_{645}w_{645} + x_{646}w_{646} + x_{647}w_{647} + x_{648}w_{648} + x_{649}w_{649} + x_{650}w_{650} + x_{651}w_{651} + x_{652}w_{652} + x_{653}w_{653} + x_{654}w_{654} + x_{655}w_{655} + x_{656}w_{656} + x_{657}w_{657} + x_{658}w_{658} + x_{659}w_{659} + x_{660}w_{660} + x_{661}w_{661} + x_{662}w_{662} + x_{663}w_{663} + x_{664}w_{664} + x_{665}w_{665} + x_{666}w_{666} + x_{667}w_{667} + x_{668}w_{668} + x_{669}w_{669} + x_{670}w_{670} + x_{671}w_{671} + x_{672}w_{672} + x_{673}w_{673} + x_{674}w_{674} + x_{675}w_{675} + x_{676}w_{676} + x_{677}w_{677} + x_{678}w_{678} + x_{679}w_{679} + x_{680}w_{680} + x_{681}w_{681} + x_{682}w_{682} + x_{683}w_{683} + x_{684}w_{684} + x_{685}w_{685} + x_{686}w_{686} + x_{687}w_{687} + x_{688}w_{688} + x_{689}w_{689} + x_{690}w_{690} + x_{691}w_{691} + x_{692}w_{692} + x_{693}w_{693} + x_{694}w_{694} + x_{695}w_{695} + x_{696}w_{696} + x_{697}w_{697} + x_{698}w_{698} + x_{699}w_{699} + x_{700}w_{700} + x_{701}w_{701} + x_{702}w_{702} + x_{703}w_{703} + x_{704}w_{704} + x_{705}w_{705} + x_{706}w_{706} + x_{707}w_{707} + x_{708}w_{708} + x_{709}w_{709} + x_{710}w_{710} + x_{711}w_{711} + x_{712}w_{712} + x_{713}w_{713} + x_{714}w_{714} + x_{715}w_{715} + x_{716}w_{716} + x_{717}w_{717} + x_{718}w_{718} + x_{719}w_{719} + x_{720}w_{720} + x_{721}w_{721} + x_{722}w_{722} + x_{723}w_{723} + x_{724}w_{724} + x_{725}w_{725} + x_{726}w_{726} + x_{727}w_{727} + x_{728}w_{728} + x_{729}w_{729} + x_{730}w_{730} + x_{731}w_{731} + x_{732}w_{732} + x_{733}w_{733} + x_{734}w_{734} + x_{735}w_{735} + x_{736}w_{736} + x_{737}w_{737} + x_{738}w_{738} + x_{739}w_{739} + x_{740}w_{740} + x_{741}w_{741} + x_{742}w_{742} + x_{743}w_{743} + x_{744}w_{744} + x_{745}w_{745} + x_{746}w_{746} + x_{747}w_{747} + x_{748}w_{748} + x_{749}w_{749} + x_{750}w_{750} + x_{751}w_{751} + x_{752}w_{752} + x_{753}w_{753} + x_{754}w_{754} + x_{755}w_{755} + x_{756}w_{756} + x_{757}w_{757} + x_{758}w_{758} + x_{759}w_{759} + x_{760}w_{760} + x_{761}w_{761} + x_{762}w_{762} + x_{763}w_{763} + x_{764}w_{764} + x_{765}w_{765} + x_{766}w_{766} + x_{767}w_{767} + x_{768}w_{768} + x_{769}w_{769} + x_{770}w_{770} + x_{771}w_{771} + x_{772}w_{772} + x_{773}w_{773} + x_{774}w_{774} + x_{775}w_{775} + x_{776}w_{776} + x_{777}w_{777} + x_{778}w_{778} + x_{779}w_{779} + x_{780}w_{780} + x_{781}w_{781} + x_{782}w_{782} + x_{783}w_{783} + x_{784}w_{784} + x_{785}w_{785} + x_{786}w_{786} + x_{787}w_{787} + x_{788}w_{788} + x_{789}w_{789} + x_{790}w_{790} + x_{791}w_{791} + x_{792}w_{792} + x_{793}w_{793} + x_{794}w_{794} + x_{795}w_{795} + x_{796}w_{796} + x_{797}w_{797} + x_{798}w_{798} + x_{799}w_{799} + x_{800}w_{800} + x_{801}w_{801} + x_{802}w_{802} + x_{803}w_{803} + x_{804}w_{804} + x_{805}w_{805} + x_{806}w_{806} + x_{807}w_{807} + x_{808}w_{808} + x_{809}w_{809} + x_{810}w_{810} + x_{811}w_{811} + x_{812}w_{812} + x_{813}w_{813} + x_{814}w_{814} + x_{815}w_{815} + x_{816}w_{816} + x_{817}w_{817} + x_{818}w_{818} + x_{819}w_{819} + x_{820}w_{820} + x_{821}w_{821} + x_{822}w_{822} + x_{823}w_{823} + x_{824}w_{824} + x_{825}w_{825} + x_{826}w_{826} + x_{827}w_{827} + x_{828}w_{828} + x_{829}w_{829} + x_{830}w_{830} + x_{831}w_{831} + x_{832}w_{832} + x_{833}w_{833} + x_{834}w_{834} + x_{835}w_{835} + x_{836}w_{836} + x_{837}w_{837} + x_{838}w_{83$$

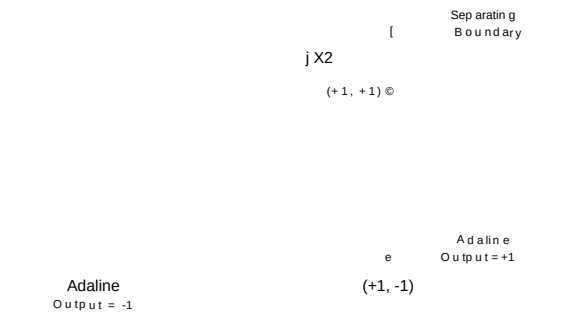


Fig. 7. Frontera de separación elíptica para realizar una función que no es linealmente separable.

Madaline se construyó con hardware [78] y se utilizó en la investigación de reconocimiento de patrones. Los pesos en esta máquina eran memristores, resistencias eléctricamente variables desarrolladas por Widrow y Hoff, que se ajustan mediante electrodeposición de un enlace resistivo [79].

Madaline se configuró de la siguiente manera. Las entradas retinianas se conectaron a una capa de elementos Adaline adaptativos, cuyas salidas se conectaron a un dispositivo lógico fijo que generaba la salida del sistema. En ese momento se desarrollaron métodos para adaptar dichos sistemas. En la Fig. 8 se muestra un ejemplo de este tipo de red. Dos Ada-

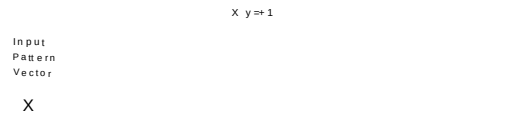


Fig. 8. Forma de Madaline con dos Adaline.

por Barron y Barron [69]-[71] y por Ivakhnenko [72] en la Unión Soviética.

El enfoque polinomial ofrece una gran simplicidad y belleza. A través de él, es posible implementar una amplia variedad de funciones discriminantes no lineales adaptativas adaptando únicamente un único elemento Adaline. Se han desarrollado varios métodos para entrenar la función discriminante polinomial. Specht desarrolló un procedimiento de entrenamiento no iterativo muy eficiente (es decir, de una sola pasada por el conjunto de entrenamiento): el método discriminante polinomial (PDM), que permite que la función discriminante polinomial implemente un clasificador no paramétrico basado en la regla de decisión de Bayes. Otros métodos para entrenar el sistema incluyen reglas iterativas de corrección de error, como las reglas del Perceptrón y a-LMS, y procedimientos iterativos de descenso de gradiente, como los algoritmos p-LMS y SER (también denominado RLS) [30]. El descenso de gradiente con un único elemento adaptativo es típicamente mucho más rápido que con una red neuronal en capas. Además, como veremos, cuando el Adaline único se entrena mediante un procedimiento de descenso de gradiente, convergerá a una solución global única.

líneas están conectadas a un dispositivo lógico AND para proporcionar una salida.

Con pesos seleccionados adecuadamente, la frontera de separación en el espacio de patrones para el sistema de la Fig. 8 sería como se muestra en la Fig. 9. Esta frontera de separación implementa la función NoR exclusiva de (5).

Después de que la función discriminante polinómica haya sido entrenada mediante un procedimiento de descenso de gradiente, los pesos del Adaline representarán una aproximación a los coeficientes en una expansión en serie de Taylor multidimensional de la función de respuesta deseada. Asimismo, si se emplean términos trigonométricos apropiados en lugar del preprocesador polinómico, la solución de pesos del Adaline aproximará los términos en la descomposición en serie de Fourier multidimensional (truncada) de una versión periódica de la función de respuesta deseada. La elección de las funciones de preprocesamiento determina qué tan bien generalizará una red para patrones fuera del conjunto de entrenamiento. Determinar funciones "buenas" sigue siendo un enfoque de la investigación actual [73], [74]. La experiencia parece indicar que, a menos que las no linealidades se elijan cuidadosamente para adaptarse al problema en cuestión, a menudo se puede obtener una mejor generalización a partir de redes con más de una capa adaptativa. De hecho, se pueden considerar las redes multicapa como redes monocapa con preprocesadores entrenables que son esencialmente auto-optimizantes.

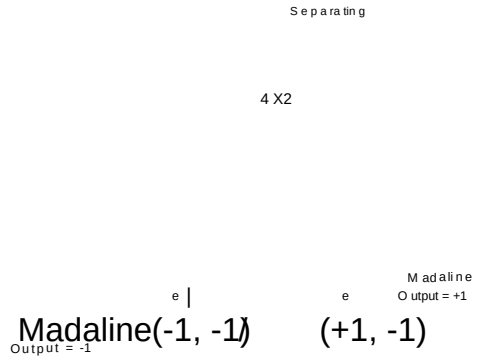


Fig. 9. Líneas de separación para la Madaline de la Fig. 8.

Las Madalines se construyeron con muchas más entradas, con muchos más elementos Adaline en la primera capa, y con diversos dispositivos lógicos fijos, como elementos AND, OR y de votación por mayoría, en la segunda capa. Estas tres funciones (Fig. 10) son todas funciones de lógica de umbral. Los valores de peso proporcionados implementarán estas tres funciones, pero la selección de los pesos no es única.

¡Claro! Aquí está la traducción al español del texto proporcionado, siguiendo todas las reglas estrictas de Madaline\*\*

Una de las primeras redes neuronales multicapa entrenables con múltiples elementos adaptativos fue la estructura Madaline de Widrow [2] y Hoff [75]. Los análisis matemáticos de Madaline fueron desarrollados en las tesis doctorales de Ridgway [76], Hoff [75] y Glanz [77]. A principios de la década de 1960, un sistema de 1000 pesos

\*\*Redes de Prealimentación Las redes de prealimentación constituyen una de las arquitecturas fundamentales

Las Madalines de la década de 1960 tenían capas de primera adaptativas y funciones de umbral fijas en las segundas capas (de salida) [76].

AND

OR

MAJ

Fig. 10. Implementaciones de Adaline de peso fijo de las funciones lógicas AND, OR y Maj.

[46]. Las redes neuronales de prealimentación actuales suelen tener muchas capas, y generalmente todas las capas son adaptativas. Las redes de retropropagación de Rumelhart et al. [47] son quizás los ejemplos más conocidos de redes multicapa. En la Fig. 11 se ilustra una red adaptativa de prealimentación completamente conectada de tres capas. En una red en capas completamente conectada,

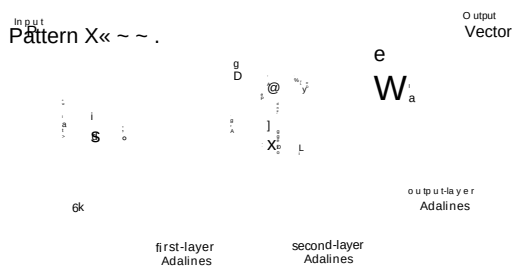


Fig. 11. Red neuronal adaptativa de tres capas.

cada Adaline recibe entradas de todas las salidas en la capa precedente.

Durante el entrenamiento, la respuesta de cada elemento de salida en la red se compara con una respuesta deseada correspondiente. Las señales de error asociadas a los elementos de salida se calculan fácilmente, por lo que la adaptación de la capa de salida es directa. La dificultad fundamental asociada con la adaptación de una red multicapa radica en obtener "señales de error" para las Adalines de la capa oculta, es decir, para las Adalines en capas distintas a la capa de salida. Los algoritmos de retropropagación y Madaline II contienen métodos para establecer estas señales de error.

En la terminología de Rumelhart et al., esto se denominaría una red de cuatro capas. Debemos enfatizar que la información aquí referida corresponde al número máximo de mapeos binarios de entrada/salida que una red puede lograr con pesos ajustados adecuadamente, no al número de bits de información que pueden almacenarse directamente en los pesos de la red.

No existe razón alguna por la cual una red de prealimentación deba poseer la estructura en capas de la Fig. 11. En el desarrollo del algoritmo de retropropagación realizado por Werbos [37], de hecho, las Adalines se ordenan y cada una recibe señales directamente de cada componente de entrada y de la salida de cada Adaline precedente. Son posibles muchas otras variaciones de la red de prealimentación. Un área interesante de la investigación actual involucra un método de retropropagación generalizado que puede utilizarse para entrenar redes de "orden superior" o "o-2" que incorporan un preprocesador polinomial para cada Adaline [47], [80].

Una característica que a menudo se desea en los problemas de reconocimiento de patrones es la invariancia de la salida de la red ante cambios en la posición y el tamaño del patrón o imagen de entrada. Se han empleado diversas técnicas para lograr invariancia ante traslación, rotación, escala y tiempo. Un método consiste en incluir en el conjunto de entrenamiento varios ejemplos de cada ejemplar transformado en tamaño, ángulo y posición, pero con una respuesta deseada que dependa únicamente del ejemplar original [78]. Otras investigaciones han abordado diversos preprocesadores basados en las transformadas de Fourier y Mellin [81], [82], así como preprocesadores neuronales [83]. Giles y Maxwell han desarrollado un ingenioso enfoque de promediado que elimina dependencias no deseadas de los términos polinomiales en unidades de lógica de umbral de alto orden (funciones discriminantes polinomiales) [74] y en redes neuronales de alto orden [80]. Otros enfoques han considerado los momentos de Zernike [84], el emparejamiento de grafos [85], los detectores de características repetidos espacialmente [9] y las salidas promediadas en el tiempo [86].

#### Capacidad de Clasificadores No Lineales

Una consideración importante que debe abordarse al comparar diversas topologías de red se refiere a la cantidad de información que pueden almacenar. De los clasificadores no lineales mencionados anteriormente, la capacidad de patrones del Adaline controlado por un preprocesador fijo compuesto de no linealidades suaves es la más sencilla de determinar. Si las entradas al sistema están distribuidas de manera uniforme en posición, las salidas de la red de preprocesamiento estarán, en general, en posición con respecto al Adaline. Por lo tanto, las entradas al Adaline satisfarán la condición requerida en la teoría de capacidad de Adaline de Cover. En consecuencia, las capacidades de patrones determinista y estadística del sistema son esencialmente iguales a las del Adaline.

Las capacidades de las estructuras Madaline I, que utilizan tanto el elemento de mayoría como el elemento OR, fueron estimadas experimentalmente por Koford a principios de la década de 1960. Aunque las funciones lógicas que pueden realizarse con estos elementos de salida son bastante diferentes, ambos tipos de elementos producen esencialmente la misma capacidad de almacenamiento estadístico. Se determinó que el número promedio de patrones que una red Madaline puede aprender a clasificar es igual a la capacidad por Adaline multiplicada por el número de Adalines en la estructura. Por lo tanto, la capacidad estadística C es aproximadamente igual al doble del número de pesos adaptativos. Aunque la Madaline y la Adaline tienen aproximadamente la misma capacidad por peso adaptativo, sin preprocesamiento la Adaline solo puede separar conjuntos linealmente separables, mientras que la Madaline no tiene tal limitación.

Una gran cantidad de trabajo teórico y experimental se ha dirigido a determinar la capacidad tanto de las Adalines como de las redes de Hopfield [87]. Se ha dedicado un esfuerzo teórico algo menor a la capacidad de patrones de las redes prealimentación multicapa, aunque existe cierto conocimiento sobre la capacidad de las redes de dos capas. Dichos resultados son de particular interés porque la red de dos capas es sorprendentemente potente. Con un número suficiente de elementos ocultos, una red signum con dos capas puede implementar cualquier función booleana.® Igualmente impresionante es la potencia de la red sigmoide de dos capas. Dado un número suficiente de elementos Adaline ocultos, dichas redes pueden implementar cualquier mapeo continuo de entrada-salida con una precisión arbitraria [94]. Aunque las redes de dos capas son bastante potentes, es probable que algunos problemas puedan resolverse de manera más eficiente mediante redes con más de dos capas. Los mapeos de predicados de orden no finito (como el problema de conectividad [95]) a menudo pueden ser calculados por redes pequeñas que utilizan retroalimentación de señales [96].

Es fácil demostrar que la Ec. (8) también acota el número de pesos necesarios para garantizar que  $N$  patrones puedan aprenderse con probabilidad  $1/2$ , excepto que en este caso el límite inferior de  $N$ , se convierte en:  $(N_y N_p + \log_2(N_y)) / C$ , de una red signum de dos capas.

Es interesante observar que los límites de capacidad (9) abarcan la capacidad determinista para la red monocapa que comprende un banco de  $N$  Adalines. En este caso, cada Adaline tendría  $N_p/N$  pesos, por lo que el sistema tendría una capacidad de patrones determinista de  $N_p/N$ . A medida que  $N_y$  se vuelve grande, la capacidad estadística también se aproxima a  $N_p/N_y$  (para  $N$  finito). Hasta que se desarrolle una teoría más completa sobre la capacidad de las redes de prealimentación, parece razonable utilizar los resultados de capacidad de la red monocapa para estimar la de las redes multicapa.

A mediados de la década de 1960, Cover estudió la capacidad de una red de signo con prealimentación que posee un número arbitrario de capas y un único elemento de salida [61], [97]. Determinó una cota inferior para el número mínimo de pesos  $(N_y + 1)$  necesario para que dicha red pueda realizar cualquier función booleana definida sobre un conjunto arbitrario de  $(N_p)$  patrones en posición general. Recientemente, Baum extendió el resultado de Cover a redes con múltiples salidas y también utilizó un argumento de construcción para encontrar las cotas superiores correspondientes para el caso especial de la red de signo de dos capas [98]. Considérese una red de prealimentación completamente conectada de dos capas compuesta por Adalines de signo, que tiene  $(N_p)$  componentes de entrada (excluyendo las entradas de sesgo) y  $(N_o)$  componentes de salida. Si se requiere que esta red aprenda a mapear cualquier conjunto que contenga  $(N_p)$  patrones en posición general a cualquier conjunto de vectores de respuesta deseado binarios (con  $(N_o)$  componentes), se deduce de los resultados de Baum que el número mínimo requerido de pesos  $(N_w)$  puede acotarse mediante

$$1 + \log_2(N_o) \leq N_w \leq N_p(N_o + 1) \quad \text{et } N_y \leq N_p \quad (8)$$

A partir de la Ec. (8), se puede demostrar que, para una red de prealimentación bicapa con un número de entradas y elementos ocultos varias veces mayor que el de salidas (por ejemplo, al menos 5 veces mayor), la capacidad de patrones determinista está acotada inferiormente por un valor ligeramente menor que  $(N_w / N_o)$ . También se deduce de la Ec. (8) que la capacidad de patrones de cualquier red de prealimentación con una relación grande de pesos a salidas (es decir,  $(N_w / N_o)$  de al menos varios miles) puede acotarse superiormente por un número algo mayor que  $(N_w / N_o) \log_2(N_w / N_o)$ . Por lo tanto, la capacidad de patrones determinista  $(N_D)$  de una red bicapa puede acotarse mediante

$$N_p, \quad N_o, \quad N_w$$

Se sabe poco acerca del número de patrones binarios que las redes sigmoideas multicapa pueden aprender a clasificar correctamente. La capacidad de patrones de las redes sigmoideas no puede ser menor que la de las redes signo de igual tamaño, sin embargo, porque a medida que los pesos de una red sigmoidea crecen hacia el infinito, esta se vuelve equivalente a una red signo con un vector de pesos en la misma dirección. Se puede extraer información relevante sobre las capacidades y los principios de funcionamiento de las redes sigmoideas a partir de la literatura [99].

La capacidad de una red tiene poca utilidad a menos que esté acompañada de generalizaciones útiles para patrones no presentados durante el entrenamiento. De hecho, si no se requiere generalización, simplemente podemos almacenar las asociaciones en una tabla de consulta y tendremos poca necesidad de una red neuronal. La relación entre la generalización y la capacidad de patrones representa un compromiso fundamental en las aplicaciones de redes neuronales: la incapacidad de la Adaline para realizar todas las funciones es, en cierto sentido, una fortaleza más que un defecto fatal imaginado por algunos críticos de las redes neuronales [95], porque ayuda a limitar la capacidad del dispositivo y, por lo tanto, mejora su capacidad de generalización.

Para lograr una buena generalización, el conjunto de entrenamiento debe contener un número de patrones al menos varias veces mayor que la capacidad de la red (es decir,  $(N_p \log N_w / N_y)$ ). Esto puede entenderse intuitivamente al notar que si el número de grados de libertad en una red (es decir,  $(N_w)$ ) es mayor que el número de restricciones asociadas con la función de respuesta deseada (es decir,  $(N_y N_p)$ ), el procedimiento de entrenamiento no podrá restringir completamente los pesos en la red. Aparentemente, esto permite que los efectos de las condiciones iniciales de los pesos interfieran con la información aprendida y degraden la capacidad de la red entrenada para generalizar. En [102] se describe un análisis detallado del rendimiento de generalización de las redes signum en función del tamaño del conjunto de entrenamiento.

Aquí está la traducción al español del texto proporcionado, siguiendo estrictamente todas las reglas especificadas.

®Esto puede observarse al notar que cualquier función booleana puede expresarse en la forma de suma de productos [91], y que dicha expresión puede realizarse con una red de dos capas utilizando las Adalines de la primera capa para implementar compuertas Y, mientras que las Adalines de la segunda capa se emplean para implementar compuertas O.

En realidad, la red puede adoptar una estructura de prealimentación arbitraria y no necesita estar organizada en capas.

El límite superior utilizado aquí es el límite laxo de Baum: número mínimo de nodos ocultos:  $N_o, [N_p/N_o] < N_i(N_p/N_o + 1)$ .

Las redes neuronales se han utilizado con éxito en una amplia gama de aplicaciones. Para obtener una visión sobre cómo se entrenan las redes neuronales y para qué pueden emplearse en el cálculo, resulta instructivo considerar la demostración de NETalk de Sejnowski y Rosenberg en 1986 [51], [52]. Con la excepción del trabajo sobre el problema del viajante de comercio con redes de Hopfield [103], esta fue la primera red neuronal



aplicación desde la década de 1960 para atraer una atención generalizada. NETtalk es una red sigmoide prealimentada de dos capas con 80 Adalines en la primera capa y 26 Adalines en la segunda capa. La red se entrena para convertir texto en habla fonéticamente correcta, una tarea muy adecuada para la implementación neuronal. La pronunciación de la mayoría de las palabras sigue reglas generales basadas en la ortografía y el contexto de la palabra, pero existen muchas excepciones y casos especiales. En lugar de programar un sistema para responder adecuadamente a cada caso, la red puede aprender las reglas generales y los casos especiales mediante ejemplos.

La Regla III también podría utilizarse. Asimismo, si la red sigmoide se reemplazara por una red de signo similar, la Regla Madaline [I] también funcionaría, aunque probablemente se necesitarían más Adalines en la primera capa para obtener un rendimiento comparable.

El resto de este artículo desarrolla y compara diversos algoritmos adaptativos para entrenar Adalines y redes neuronales artificiales con el fin de resolver problemas de clasificación, como NETalk. Estos mismos algoritmos pueden utilizarse para entrenar redes en otros problemas, como aquellos que involucran control no lineal [50], identificación de sistemas [50], [104], procesamiento de señales [30] o toma de decisiones [55].

Una de las características más notables de NETtalk es que aprende a pronunciar palabras en etapas que recuerdan al proceso de aprendizaje en los niños. Cuando la salida de NETtalk se conecta a un sintetizador de voz, el sistema produce sonidos de balbuceo durante las primeras etapas del proceso de entrenamiento. A medida que la red aprende, a continuación domina las reglas generales y, como un niño, tiende a cometer muchos errores al aplicar estas reglas incluso cuando no son apropiadas. Sin embargo, a medida que el entrenamiento continúa, la red eventualmente abstrae las excepciones y los casos especiales, y es capaz de producir un habla inteligible con pocos errores.

El funcionamiento de NETtalk es sorprendentemente simple. Su entrada es un vector de siete caracteres (incluyendo espacios) proveniente de una transcripción de texto, y su salida es información fonética correspondiente a la pronunciación del carácter central (el cuarto) en el campo de entrada de siete caracteres. Los otros seis caracteres proporcionan contexto, lo que ayuda a determinar el fonema deseado. Para leer texto, la ventana de siete caracteres se desplaza a lo largo de un documento en la memoria de la computadora y la red genera una secuencia de símbolos fonéticos que pueden utilizarse para controlar un sintetizador de voz. Cada uno de los siete caracteres en la entrada de la red es un vector binario de 29 componentes, donde cada componente representa un carácter alfabético o signo de puntuación diferente. Se coloca un uno en el componente asociado con el carácter representado; todos los demás componentes se establecen en cero.

Las 26 salidas del sistema corresponden a 23 características articulatorias y 3 características adicionales que codifican el acento y los límites silábicos. Al entrenar la red, el vector de respuesta deseada tiene ceros en todos los componentes, excepto aquellos que corresponden a las características fonéticas asociadas con el carácter central en el campo de entrada. En un experimento, Sejnowski y Rosenberg hicieron que el sistema escaneara una transcripción de 1024 palabras de habla continua transcrita fonéticamente. Con la presentación de cada ventana de siete caracteres, los pesos del sistema se entrenaron mediante el algoritmo de retropropagación en respuesta al error de salida de la red. Después de aproximadamente 50 presentaciones de todo el conjunto de entrenamiento, la red fue capaz de producir habla precisa a partir de datos a los que la red no había estado expuesta durante el entrenamiento.

Aquí está la traducción al español académico formal, siguiendo todas las reglas especificadas: la retropropagación no es la única técnica que podría utilizarse para entrenar NETtalk. En otros experimentos, se empleó el método más lento de aprendizaje de Boltzmann y, de hecho, Madal-

## II. ADAPTACIÓN- EL PRINCIPIO DE MÍNIMA PERTURBACIÓN

Los algoritmos iterativos descritos en este artículo están diseñados de acuerdo con un único principio subyacente. Estas técnicas son dos algoritmos LMS, las reglas de Mays y el procedimiento del Perceptrón para entrenar un Adaline simple, la regla MRII para entrenar la Madaline simple, así como las técnicas MRII, MRIII y de retropropagación para entrenar Madalines multicapa basadas todas en el principio de mínima perturbación: adaptarse para reducir el error de salida para el patrón de entrenamiento actual, con una mínima perturbación a las respuestas ya aprendidas. A menos que se practique este principio, es difícil almacenar simultáneamente las respuestas de los patrones requeridos. El principio de mínima perturbación es intuitivo. Fue la idea motivadora que condujo al descubrimiento del algoritmo LMS y las reglas de la Madaline. De hecho, el algoritmo LMS existió durante varios meses como una regla de reducción de error antes de que se descubriera que el algoritmo utilizaba un gradiente instantáneo para seguir la trayectoria de descenso más pronunciado y minimizar el error cuadrático medio del conjunto de entrenamiento. Fue entonces cuando se le dio el nombre de algoritmo "LMS" (mínimos cuadrados medios).

## IV. REGLAS DE CORRECCIÓN DE ERROR ELEMENTO DE UMBRAL MONOCAPA

A medida que evolucionaron los algoritmos adaptativos, surgieron principalmente dos tipos de reglas en línea. Las reglas de corrección de error modifican los pesos de una red para corregir el error en la respuesta de salida ante el patrón de entrada actual. Las reglas de gradiente modifican los pesos de una red durante cada presentación de patrón mediante descenso de gradiente, con el objetivo de reducir el error cuadrático medio, promediado sobre todos los patrones de entrenamiento. Ambos tipos de reglas emplean procedimientos de entrenamiento similares. Sin embargo, debido a que se basan en objetivos distintos, pueden presentar características de aprendizaje significativamente diferentes.

Las reglas de corrección de error, por necesidad, tienden a menudo a ser \*ad hoc\*. Se utilizan con mayor frecuencia cuando los objetivos de entrenamiento no se cuantifican fácilmente, o cuando un problema no se presta a un análisis tratable. Una aplicación común, por ejemplo, concierne al entrenamiento de redes neuronales que contienen funciones discontinuas. Una excepción es el algoritmo a-LMS, una regla de corrección de error que ha demostrado ser una técnica extremadamente útil para encontrar soluciones a problemas lineales bien definidos y tratables.

Comenzamos con las reglas de corrección de error aplicadas inicialmente a elementos Adaline individuales, y luego a redes de Adalines.

### A. Reglas Lineales

Las reglas lineales de corrección de error modifican los pesos del elemento de umbral adaptativo con cada presentación de patrón para realzar una corrección de error proporcional al error mismo. La regla lineal, a-LMS, se describe a continuación.

"The input representation often has a considerable impact on the success of a network. [In NETtalk, the inputs are sparsely coded in 29 component sets. One might consider instead choosing a 5-bit binary representation of the 7-bit ASCII code. It should be clear, however, that in this case the sparse representation helps simplify the network's job of interpreting inputs in put characters as 29 distinct symbols. Usually the appropriate input encoding is not difficult to decide. When intuition fails, however, one sometimes must experiment with different encodings to find one that works well.

El algoritmo a-LMS: El algoritmo a-LMS, o regla delta de Widrow-Hoff, aplicado a la adaptación de un Adaline individual (Fig. 2) incorpora el principio de perturbación mínima. La ecuación de actualización de pesos para la forma original del algoritmo puede expresarse como

$$W_{k+1} = W_k + a e_k X_k \quad (10)$$

El índice de tiempo o número de ciclo de adaptación es  $k$ .  $W_{k+1}$  es el siguiente valor del vector de pesos,  $W_k$  es el valor actual del vector de pesos,  $X_k$  es el vector del patrón de entrada actual. El error lineal actual  $e_k$  se define como la diferencia entre la respuesta deseada  $d_k$  y la salida lineal  $s_k = W_k^T X_k$  antes de la adaptación:

$$e_k = d_k - s_k = d_k - W_k^T X_k \quad (11)$$

Cambiar los pesos produce un cambio correspondiente en el:

$$\Delta W_k = a (d_k - W_k^T X_k) X_k = a e_k X_k \quad (12)$$

De acuerdo con la regla a-LMS de la Ec. (10), el cambio de peso es el siguiente:

$$W_{k+1} = W_k + \Delta W_k = W_k + a e_k X_k \quad (13)$$

Combinando las Ecs. (12) y (13), obtenemos

$$\Delta W_k = a X_k e_k = a e_k X_k \quad (14)$$

Por lo tanto, el error se reduce en un factor de  $\alpha$  a medida que los pesos se modifican mientras se mantiene fijo el patrón de entrada. La presentación de un nuevo patrón de entrada inicia el siguiente ciclo de adaptación. El siguiente error se reduce entonces en un factor de  $\alpha$ , y el proceso continúa. El vector de pesos inicial suele elegirse como cero y se adapta hasta alcanzar la convergencia. En entornos no estacionarios, los pesos generalmente se adaptan de forma continua.

La elección de  $\alpha$  controla la estabilidad y la velocidad de convergencia [30]. Para vectores de patrones de entrada independientes en el tiempo, la estabilidad está garantizada para la mayoría de los propósitos prácticos si

$$0 < \alpha < 2 \quad (15)$$

Hacer que  $\alpha$  sea mayor que 1 generalmente no tiene sentido, ya que el error se corregiría en exceso. La corrección total del error se produce con  $\alpha = 1$ . Un rango práctico para  $\alpha$  es

$$0.1 < \alpha < 1.0 \quad (16)$$

Este algoritmo es autónomo en el sentido de que la elección de  $\alpha$  no depende de la magnitud de las señales de entrada. La actualización del vector de pesos es colineal con el patrón de entrada y de una magnitud inversamente proporcional a  $\|X_k\|^2$ . Con entradas binarias  $\pm 1$ ,  $\|X_k\|^2$  es igual al número de pesos y no varía de un patrón a otro. Si las entradas binarias son los valores habituales 1 y 0, no se produce adaptación para los pesos con entradas 0, mientras que con entradas  $\pm 1$ , todos los pesos se adaptan en cada ciclo y la convergencia tiende a ser más rápida. Por esta razón, las entradas simétricas  $\pm 1$  y  $-1$  son generalmente preferidas.

La Figura 12 proporciona una representación geométrica de cómo funciona la regla a-LMS. De acuerdo con la Ec. (13),  $W_{k+1}$  es igual a  $W_k$  más  $\Delta W_k$ , y  $\Delta W_k$  es paralelo al vector del patrón de entrada  $X_k$ . A partir de la Ec. (12), el cambio en el error es igual al producto punto negativo de  $X_k$  y  $\Delta W_k$ . Dado que el algoritmo a-LMS

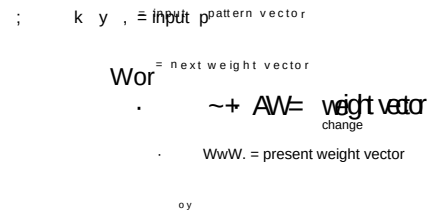


Fig. 12. Corrección de pesos mediante la regla LMS.

selecciona  $W_k$  para que sea colineal con  $X_k$ , logrando así la corrección de error deseada con un cambio de peso de la menor magnitud posible. Al adaptarse para responder adecuadamente a un nuevo patrón de entrada, las respuestas a patrones de entrenamiento previos se ven mínimamente perturbadas, en promedio.

El algoritmo a-LMS corrige el error  $y$ , si todos los patrones de entrada tienen la misma longitud, minimiza el error cuadrático medio [30]. Este algoritmo es más conocido por dicha propiedad.

#### B. Reglas No Lineales

El algoritmo a-LMS es una regla lineal que realiza correcciones de error proporcionales al error. Se sabe [105] que, en algunos casos, esta regla lineal puede no lograr separar patrones de entrenamiento que son linealmente separables. Cuando esto genera dificultades, pueden emplearse reglas no lineales. En las siguientes secciones, describimos reglas no lineales tempranas, desarrolladas por Rosenblatt [106], [5] y Mays [105]. Estas reglas no lineales también realizan cambios en el vector de pesos colineales con el vector del patrón de entrada (la dirección que provoca la mínima perturbación), cambios que se basan en el error lineal pero no son directamente proporcionales a este.

Aquí está la traducción al español académico formal, siguiendo todas las reglas especificadas: La Regla de Aprendizaje del Perceptrón: El perceptrón-alfa de Rosenblatt [106], [5], diagramado en la Fig. 13, procesa patrones de entrada

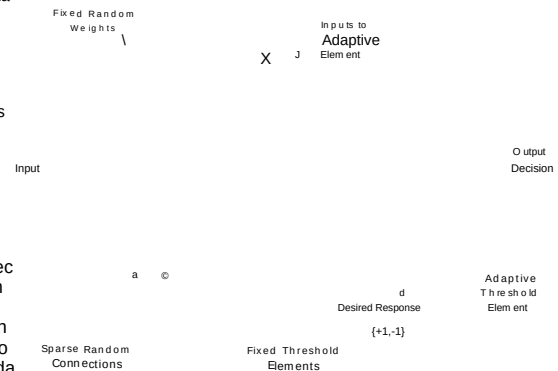


Fig. 13. a-Perceptrón de Rosenblatt.

términos con una primera capa de dispositivos lógicos fijos escasamente conectados de forma aleatoria. Las salidas de la primera capa fija alimentaban una segunda capa, que consistía en un único elemento lineal adaptativo de umbral. A parte de la convención de que sus señales de entrada eran binarias  $\{1, 0\}$  y de que no se incluía un peso de sesgo, este elemento es equivalente al elemento Adaline. La regla de aprendizaje para el a-Perceptrón es muy similar a la de LMS, pero su comportamiento es, de hecho, bastante diferente.

Es interesante señalar que la regla de aprendizaje del Perceptrón de Rosenblatt se presentó por primera vez en 1960 [106], y la regla LMS de Widrow y Hoff se presentó por primera vez el mismo año, unos meses después [59]. Estas reglas fueron desarrolladas de forma independiente en 1959.

El elemento de umbral adaptativo del a-Perceptrón se muestra en la Fig. 14. La adaptación mediante la regla del Perceptrón hace que

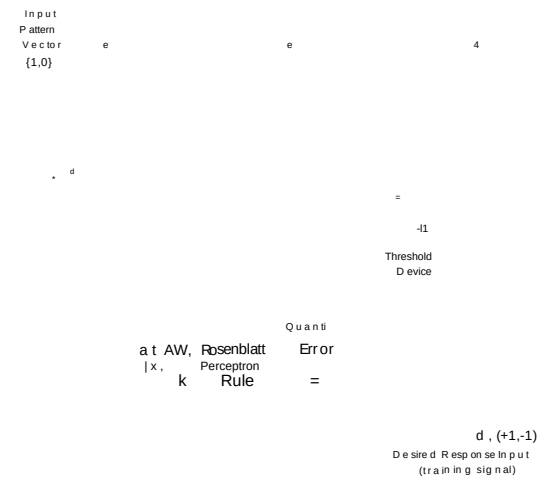


Fig. 14. El elemento de umbral adaptativo del Perceptrón.

uso del "error del cuantizador" ( $\epsilon$ ), definido como la diferencia entre la respuesta deseada y la salida del cuantizador.

La regla del Perceptrón, denominada en ocasiones procedimiento de convergencia del Perceptrón, no adapta los pesos si la decisión de salida es correcta, es decir, si  $\epsilon = 0$ . Sin embargo, si la decisión de salida discrepa con la respuesta binaria deseada ( $d$ ), la adaptación se efectúa sumando el vector de entrada al vector de pesos cuando el error ( $e$ ) es positivo, o restando el vector de entrada del vector de pesos cuando el error ( $e$ ) es negativo. De este modo, se añade al vector de pesos la mitad del producto del vector de entrada y el error del cuantificador ( $e$ ). La regla del Perceptrón es idéntica al algoritmo ( $\alpha$ )-LMS, excepto que en la regla del Perceptrón se utiliza la mitad del error del cuantificador ( $e/2$ ) en lugar del error lineal normalizado ( $e/\|X\|$ ). La regla del Perceptrón es no lineal, en contraste con la regla LMS, que es lineal (compárense las Figs. 2 y 14). No obstante, la regla del Perceptrón puede expresarse en una forma muy similar a la regla ( $\alpha$ )-LMS de la Ec. (10):

$$W_i = W_e + \frac{e}{2} X_i \quad (18)$$

Rosenblatt normalmente establecía  $a$  en uno. En contraste con a-LMS, la elección de  $a$  no afecta la estabilidad del algoritmo del Perceptrón, y solo influye en el tiempo de convergencia si el vector de pesos inicial es distinto de cero. Asimismo, mientras que a-LMS puede utilizarse con respuestas deseadas tanto analógicas como binarias, la regla de Rosenblatt solo puede emplearse con respuestas deseadas binarias.

El perceptrón tiene su adaptación cuando los patrones de entrenamiento se separan correctamente. Sin embargo, no existe una fuerza restrictiva que controle la magnitud de los pesos. La dirección del vector de pesos, y no su magnitud, determina

la función de decisión. Se ha demostrado que la regla del Perceptrón es capaz de separar cualquier conjunto de patrones de entrenamiento linealmente separable [5], [107], [46], [105]. Si los patrones de entrenamiento no son linealmente separables, el algoritmo del Perceptrón continúa indefinidamente y, a menudo, no produce una solución de bajo error, incluso si existe alguna. En la mayoría de los casos, si el conjunto de entrenamiento no es separable, el vector de pesos tiende a gravitar hacia cero, de modo que incluso si ( $a$ ) es muy pequeño, cada adaptación puede afectar drásticamente la función de conmutación implementada por el Perceptrón.

Este comportamiento es muy diferente al del algoritmo a-LMS. El uso continuado de a-LMS no conduce a una solución de pesos no razonable si el conjunto de patrones no es linealmente separable. Sin embargo, tampoco se garantiza que este algoritmo separe cualquier conjunto de patrones linealmente separable. a-LMS típicamente se aproxima a lograr dicha separación, pero su objetivo es diferente: la reducción del error en la salida lineal del elemento adaptativo.

Rosenblatt también introdujo variantes de la regla de incremento fijo que hemos discutido hasta ahora. Una popular fue la versión de corrección absoluta de la regla del Perceptrón.<sup>1</sup> Esta regla es idéntica a la establecida en la Ec. (18), excepto que el tamaño del incremento ( $a$ ) se elige en cada presentación como el entero más pequeño que corrige el error de salida en una sola presentación. Si el conjunto de entrenamiento es separable, esta variante posee todas las características de la versión de incremento fijo con ( $a$ ) establecido en 1, excepto que generalmente alcanza una solución en menos presentaciones.

**\*\*Algoritmos de Mays:\*\*** En su tesis doctoral [105], Mays describió un a regla de "adaptación por incremento" y una regla de "adaptación por relajación modificada". La versión de incremento fijo de la regla del Perceptrón es un caso especial de la regla de adaptación por incremento.

La adaptación incremental en su forma general implica el uso de una "zona muerta" para la salida lineal ( $s$ ), igual a ( $\pm \gamma$ ) alrededor de cero. Todas las respuestas deseadas son ( $\pm 1$ ) (consulte la Fig. 14). Si la salida lineal ( $s$ ) cae fuera de la zona muerta ( $|s| > \gamma$ ), la adaptación sigue una variante normalizada de la regla del Perceptrón de incremento fijo (utilizando ( $\alpha/\|X\|^2$ ) en lugar de ( $\alpha$ )). Si la salida lineal cae dentro de la zona muerta, independientemente de si la respuesta de salida ( $y$ ) es correcta o no, los pesos se adaptan mediante la variante normalizada de la regla del Perceptrón como si la respuesta de salida ( $y$ ) hubiera sido incorrecta. La regla de actualización de pesos para el algoritmo de adaptación incremental de Mays puede expresarse matemáticamente como

$$W_i = W_i + a \frac{\epsilon}{\|X\|} X_i \quad \text{if } |s| = y \quad (19)$$

donde ( $\epsilon$ ) es el error del cuantificador de la Ec. (17).

Con la zona muerta ( $\gamma > 0$ ), el algoritmo de adaptación incremental de Mays se reduce a una versión normalizada de la regla de aprendizaje del perceptrón.

Esto ocurre porque la longitud del vector de pesos disminuye con cada adaptación que no provoca que la salida lineal ( $s$ ) cambie de signo y asuma una magnitud mayor que la anterior a la adaptación. Aunque existen excepciones, para la mayoría de los problemas esta situación ocurre solo raramente si el vector de pesos es mucho más largo que el vector de incremento de pesos.

Los términos incremento fijo y corrección absoluta se deben a Nilsson [46]. Rosenblatt se refirió a métodos de estos tipos como reglas de aprendizaje cuantizadas y no cuantizadas, respectivamente.

La regla de adaptación por incremento fue propuesta por otros antes que Mays, aun que desde una perspectiva diferente [107].

regla del perceptrón (18). Mays demostró que, si los patrones de entrenamiento son linealmente separables, la adaptación por incremento siempre convergerá y separará los patrones en un número finito de pasos. También mostró que el uso de la zona muerta reduce la sensibilidad a los errores de peso. Si el conjunto de entrenamiento no es linealmente separable, la regla de adaptación por incremento de Mays suele funcionar mucho mejor que la regla del perceptrón, ya que una zona muerta suficientemente grande tiende a hacer que el vector de pesos se adapte alejándose de cero cuando existe alguna solución razonablemente buena. En tales casos, el vector de pesos puede parecer a veces que deambula sin rumbo, pero normalmente permanecerá en una región asociada con un promedio relativamente bajo.

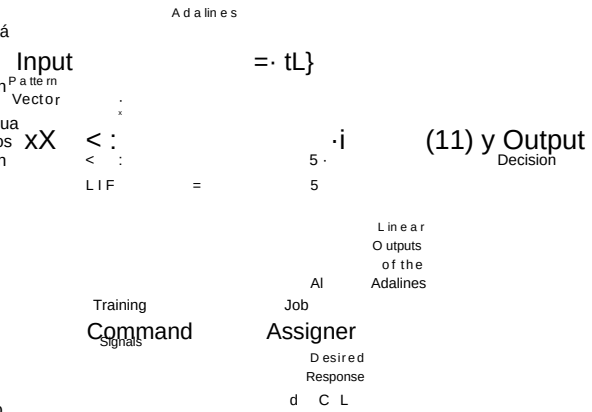


Fig. 15. Un ejemplo de cinco Adalines de la arquitectura Madaline

La regla de adaptación por incremento modifica los pesos con incrementos que, en general, no son proporcionales al error lineal  $\epsilon$ . La otra regla de Mays, la adaptación modificada, se aproxima más al algoritmo a-LMS en su uso del error lineal  $\epsilon$  (véase la Fig. 2). La respuesta deseada y los niveles de salida del cuantificador son binarios: 1. Si la salida del cuantificador  $\hat{y}$  es incorrecta, o si la salida lineal  $y$  cae dentro de la zona muerta, la adaptación sigue el algoritmo a-LMS para reducir el error lineal. Si la salida del cuantificador  $\hat{y}$  es correcta y la salida lineal  $y$  se encuentra fuera de la zona muerta, los pesos no se adaptan. La regla de actualización de pesos para este algoritmo puede expresarse como

AND gate, o el tomador de decisión por mayoría discutido previamente. Los pesos de los Adalines se inicializan con valores aleatorios pequeños.

$$W_{ii1} = \begin{cases} W_{ii1} + \alpha \epsilon_i & \text{if } \epsilon_i = 0 \text{ and } |s_i| = y_i \\ W_{ii1} & \text{otherwise} \end{cases} \quad (20)$$

donde  $\epsilon_i$  es el error del cuantificador de la Ec. (17).

Si la zona muerta  $\Delta$  se establece en 0, este algoritmo se reduce al algoritmo a-LMS (10). Mays demostró que, para una zona muerta  $\Delta = 0$ ,  $\Delta$  (gamma = 1) y una tasa de aprendizaje  $\alpha$  ( $0 < \alpha < 2$ ), este algoritmo convergerá y separará cualquier conjunto de entrada linealmente separable en un número finito de pasos. Si el conjunto de entrenamiento no es linealmente separable, este algoritmo se comporta de manera muy similar a la regla de adaptación incremental de Mays.

Los dos algoritmos de Mays logran resultados similares de separación de patrones. La elección de  $\Delta$  no afecta la estabilidad, aunque sí influye en el tiempo de convergencia. Las dos reglas difieren en sus propiedades de convergencia, pero no existe consenso sobre cuál es el mejor algoritmo. Algoritmos como estos pueden ser bastante útiles, y creemos que aún quedan muchos más por inventar y analizar.

El algoritmo a-LMS, el procedimiento del Perceptrón y los algoritmos de Mays pueden utilizarse todos para adaptar el elemento Adaline individual, o pueden incorporarse en procedimientos para adaptar redes de dichos elementos. Los procedimientos de adaptación de redes multicapa que emplean algunos de estos algoritmos se discuten a continuación.

## V. REGLAS DE CORRECCIÓN DE ERROR REDES MULTIELEMENTO

Los algoritmos que se discuten a continuación son la regla Madaline de Widrow-Hoff de principios de la década de 1960, ahora denominada Regla Madaline I (MRI), y la Regla Madaline II (MRII), desarrollada por Widrow y Winter en 1987.

### A. Regla de Madaline

La regla MRI permite la adaptación de una primera capa de elementos Adaline de imitación dura (signo) cuyas salidas proporcionan entradas a una segunda capa, compuesta por un único elemento de lógica de umbral fijo que puede ser, por ejemplo, la puerta OR.

Diferencias en las condiciones iniciales y los resultados de la adaptación subsiguiente hacen que los diversos elementos asuman la "responsabilidad" de ciertas partes del problema de entrenamiento. El principio básico de la distribución de carga se resume así: Asignar la responsabilidad al Adaline o a los Adalines que puedan asumirla con mayor facilidad.

En la Fig. 15, el "asignador de tareas", un proceso puramente mecanizado, asigna la responsabilidad durante el entrenamiento transfiriendo los comandos de adaptación apropiados y las señales de respuesta deseada a los Adalines seleccionados. El asignador de tareas utiliza información de salida lineal. La distribución de carga es importante, ya que da como resultado que los diversos elementos adaptativos desarrollen vectores de pesos individuales. Si todos los vectores de pesos fueran iguales, no tendría sentido tener más de un elemento en la primera capa.

Al entrenar la Madaline, la secuencia de presentación de los patrones debe ser aleatoria. Al experimentar con esto, Ridgway [76] descubrió que la presentación cíclica de los patrones podía conducir a ciclos de adaptación. Estos ciclos provocarían que los pesos de toda la Madaline entraran en un ciclo, impidiendo la convergencia.

El sistema adaptativo de la Fig. 15 fue sugerido por el sentido común y demostró funcionar bien en simulaciones. Ridgway descubrió que la probabilidad de que un Adaline dado sea adaptado en respuesta a un patrón de entrada es mayor si ese elemento había asumido dicha responsabilidad durante el ciclo de adaptación anterior, cuando el patrón se presentó más recientemente. La división de responsabilidades se estabiliza al mismo tiempo que las respuestas de los elementos individuales se estabilizan en su parte de la carga. Cuando el problema de entrenamiento no es perfectamente separable por este sistema, el proceso de adaptación tiende a minimizar la probabilidad de error, aunque es posible que el algoritmo se "estancue" en óptimos locales.

La estructura Madaline de la Fig. 15 consta de 2 capas: la primera capa está compuesta por elementos lógicos adaptativos, y la segunda por lógica fija. Se podría emplear una variedad de dispositivos de lógica fija para la segunda capa. Hoff [75] desarrolló diversas reglas de adaptación MRI que pueden utilizarse con todos los posibles elementos de salida de lógica fija. Un procedimiento de entrenamiento fácil de describir se obtiene cuando el elemento de salida es una puerta OR. Durante el entrenamiento, si la salida deseada para un patrón de entrada dado es 1, solo se adaptaría el Adaline cuya salida lineal esté más cercana a cero, en caso de que se requiera alguna adaptación; es decir, si todos los Adalines producen salidas de 1. Si la salida deseada es 1, todos los elementos deben generar salidas de 1, y cualquier elemento que produzca salidas de 0 debe ser adaptado.

La regla MRI obedece al «principio de perturbación mínima» en el siguiente sentido. No se adaptan más elementos Adaline de los necesarios para corregir la decisión de salida y cualquier restricción de zona muerta. Se adaptan los elementos cuyas salidas lineales están más próximas a cero, ya que requieren los cambios de peso más pequeños para invertir sus respuestas de salida. Además, siempre que se adapta un Adaline, los pesos se modifican en la dirección de su vector de entrada, proporcionando la corrección de error necesaria con un cambio de peso mínimo.

## B. Regla Madaline I

La regla MRI fue recientemente extendida para permitir la adaptación de redes binarias multicapa por Winter y Widrow con la introducción de la Regla Madaline II (MRII) [43], [83], [108]. En la Fig. 16 se muestra una red MRI típica de dos capas. Los pesos en ambas capas son adaptativos.

El entrenamiento con la regla MRII es similar al entrenamiento con el algoritmo MRI. Los pesos se inicializan con valores aleatorios pequeños. Los patrones de entrenamiento se presentan en una secuencia aleatoria. Si la red produce un error durante una presentación de entrenamiento, se comienza adaptando los Adalines de la primera capa.

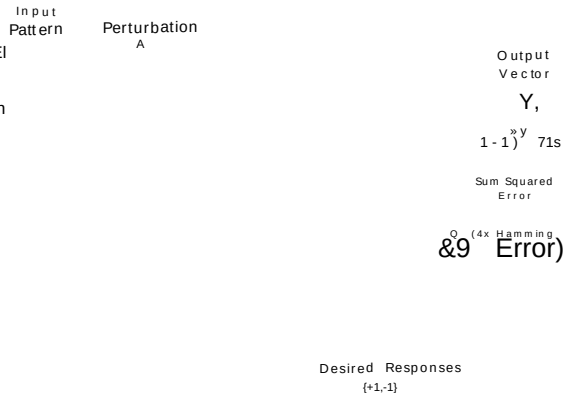


Fig. 16. Arquitectura típica de Madaline II de dos capas.

Por el principio de mínima perturbación, seleccionamos el Adaline de la primera capa con la menor magnitud de salida lineal y realizamos una "adaptación de prueba" invirtiendo su salida binaria. Esto puede realizarse sin adaptación añadiendo una perturbación  $\Delta s$  de amplitud y polaridad adecuadas a la suma del Adaline (véase la Fig. 16). Si el error de Hamming en la salida se reduce mediante esta inversión de bits, es decir, si el número de errores de salida disminuye, se elimina la perturbación  $\Delta s$  y los pesos del elemento Adaline seleccionado se modifican mediante LMS en una dirección colineal con el vector de entrada correspondiente en la dirección que refuerza la inversión de bits con una mínima perturbación en los pesos. Por el contrario, si la adaptación de prueba no mejora la respuesta de la red, no se realiza ninguna adaptación de peso.

Después de finalizar con el primer elemento, perturbamos y actualizamos otros Adalines en la primera capa que presentan magnitudes de salida lineal "suficientemente pequeñas". Se pueden lograr reducciones adicionales del error, si se desea, invirtiendo pares, triples, y así sucesivamente, hasta un límite predeterminado. Tras agotar las posibilidades con la primera capa, pasamos a la siguiente capa y procedemos de manera similar. Al alcanzar la capa final, cada uno de los elementos de salida se adapta mediante a-LMS. En este punto, se selecciona al azar un nuevo patrón de entrenamiento y se repite el procedimiento. El objetivo es reducir el error de Hamming con cada presentación, minimizando así, según lo esperado, el error de Hamming promedio sobre el conjunto de entrenamiento. Al igual que MRI, el procedimiento puede modificarse para que las adaptaciones sigan una regla de corrección absoluta o una de las reglas de Mays en lugar de a-LMS. De manera similar a MRI, MRII puede "estancarse" en óptimos locales.

## VI. REGLAS DE DESCENSO MÁS PRONUNCIADO ELEMENTO DE UMBRAL ÚNICO

Hasta ahora, hemos descrito una variedad de reglas de adaptación que actúan para reducir el error con la presentación de cada patrón de entrenamiento. A menudo, el objetivo de la adaptación es reducir el error promediado de alguna manera sobre el conjunto de entrenamiento. La función de error más común es el error cuadrático medio (MSE), aunque en algunas situaciones otros criterios de error pueden ser más apropiados [109]-[111]. Los enfoques más populares para la reducción del MSE tanto en redes monocapa como multicapa se basan en el método de descenso más pronunciado. Enfoques de gradiente más sofisticados, como las técnicas cuasi-Newton [30], [112]-[114] y de gradiente conjugado [114], [115], suelen tener mejores propiedades de convergencia, pero

las condiciones bajo las cuales se justifica la complejidad adicional no se conocen generalmente. La discusión que sigue se limita a la minimización del MSE mediante el método de descenso más pronunciado [116], [117]. Los procedimientos de aprendizaje más sofisticados suelen requerir muchos de los mismos cálculos utilizados en el procedimiento básico de descenso más pronunciado.

Definiendo el vector  $\mathbf{P}$  como la correlación cruzada entre la respuesta deseada (un escalar) y el vector  $\mathbf{X}$ , se obtiene entonces

$$\mathbf{P} = \mathbf{E}\{\mathbf{d} \mathbf{X}^T\} = \mathbf{E}\{\mathbf{d} \mathbf{X}^T\} \quad (25)$$

La matriz de correlación de entrada  $\mathbf{R}$  se define en términos del promedio de conjunto

$$\mathbf{R} = \mathbf{E}\{\mathbf{X} \mathbf{X}^T\}$$

La adaptación de una red mediante descenso más pronunciado comienza con un valor inicial arbitrario  $\mathbf{W}$  para el vector de pesos del sistema. Se mide el gradiente de la función de error cuadrático medio (MSE) y el vector de pesos se modifica en la dirección correspondiente al negativo del gradiente medido. Este procedimiento se repite, lo que provoca que el MSE se reduzca sucesivamente en promedio y que el vector de pesos se aproxime a un valor localmente óptimo.

$$\mathbf{R} = \frac{1}{N} \sum_{k=1}^N \mathbf{X}_k \mathbf{X}_k^T \quad (26)$$

El método de descenso más pronunciado puede describirse mediante la relación

$$\mathbf{W}_{k+1} = \mathbf{W}_k + \eta \mathbf{X}_k \mathbf{d}_k$$

Esta matriz es real, simétrica y definida positiva, o en casos raros, semidefinida positiva. El MSE puede, por lo tanto, expresarse como

$$\mathbf{W}^T \mathbf{W} = \mathbf{W}^T \mathbf{V} \quad (21)$$

donde  $\eta$  es un parámetro que controla la estabilidad y la tasa de convergencia, y  $\mathbf{V}$  es el valor del gradiente en un punto de la superficie de MSE correspondiente a  $\mathbf{W}$ .

$$\mathbf{E} = \mathbf{W}^T \mathbf{R} \mathbf{W} + \frac{1}{2} \mathbf{P}^T \mathbf{W} \quad (27)$$

Para comenzar, derivamos reglas para la minimización mediante descenso más pronunciado del error cuadrático medio (MSE) asociado con un elemento Adaline individual. Estas reglas se generalizan luego para aplicarse a redes neuronales completas. Al igual que las reglas de corrección de error, las reglas de descenso más pronunciado más prácticas y eficientes suelen trabajar con un patrón a la vez. Estas minimizan el error cuadrático medio, de forma aproximada, promediado sobre todo el conjunto de patrones de entrenamiento.

Nota que el MSE es una función cuadrática de los pesos. Se trata de una superficie hiperparaboloide convexa, una función que nunca toma valores negativos. La Figura 17 muestra una superficie típica del MSE.

#### A. Reglas Lineales

Aquí está la traducción al español académico formal, siguiendo estrictamente todas las reglas de descenso más pronunciado para el elemento de umbral único se consideran lineales si los cambios de peso son proporcionales al error lineal, la diferencia entre la respuesta deseada  $d$  y la salida lineal del elemento  $s$ .

**\*\*Superficie de Error Cuadrático Medio del Combinador Lineal:\*\*** En esta sección demostramos que la superficie del error cuadrático medio (MSE) del combinador lineal de la Fig. 1 es una función cuadrática de los pesos y, por lo tanto, es fácilmente recorrida mediante el descenso de gradiente.

Sea el patrón de entrada  $\mathbf{X}$  y la respuesta deseada asociada  $d$ , extraídos de una población estadísticamente estacionaria. Durante la adaptación, el vector de pesos varía de modo que, incluso con entradas estacionarias, la salida  $s$  y el error  $e$  serán generalmente no estacionarios. Se debe tener cuidado al definir el error cuadrático medio (MSE), ya que varía en el tiempo. La única posibilidad es un promedio de conjunto, definido a continuación.

En la  $k$ -ésima iteración, sea el vector de pesos  $\mathbf{W}_k$ . Elevando al cuadrado y expandiendo la ecuación (11) se obtiene

$$e = (d - \mathbf{X}^T \mathbf{W}) \quad (22)$$

$$e^2 = d^2 - 2d \mathbf{X}^T \mathbf{W} + \mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W} \quad (23)$$

Ahora considere un conjunto de combinadores lineales adaptativos idénticos, cada uno con el mismo vector de pesos  $\mathbf{W}$ , en la  $k$ -ésima iteración. Suponga que cada combinador posee entradas individuales  $\mathbf{X}$  y  $d$ , derivadas de conjuntos estacionarios ergódicos. Cada combinador generará un error individual  $e$ , representado por la Ec. (23). Promediando la Ec. (23) sobre el conjunto se obtiene

$$\mathbf{E}\{e^2\} = \mathbf{E}\{d^2\} - 2\mathbf{E}\{d \mathbf{X}^T \mathbf{W}\} + \mathbf{W}^T \mathbf{E}\{\mathbf{X} \mathbf{X}^T\} \mathbf{W} \quad (24)$$

El gradiente  $\nabla$  de la función de error cuadrático medio (MSE) con respecto a  $\mathbf{W}$  se obtiene diferenciando la Ec. (27):

$$\nabla E = 2\mathbf{R} \mathbf{W} - \mathbf{P} \quad (28)$$

$$\nabla E = 2\mathbf{R} \mathbf{W} - \mathbf{P}$$

We assume here that  $\mathbf{X}$  includes a bias component  $x_0 = +1$ .

Esta es una función lineal de los pesos. El vector de pesos óptimo  $\mathbf{W}^*$ , generalmente denominado vector de pesos de Wiener, se obtiene a partir de la Ec. (28) media a  $\mathbf{W}^*$ , la solución óptima de Wiener discutida anteriormente. Una demostración de esto puede encontrarse en [30].

$$\mathbf{W}^* = \mathbf{R}^{-1} \mathbf{P} \quad (29)$$

Esta es una forma matricial de la ecuación de Wiener-Hopf [118]-[120]. En la siguiente sección examinamos u-LMS, un algoritmo que nos permite obtener una estimación precisa de  $\mathbf{W}^*$  sin necesidad de calcular primeramente  $\mathbf{R}$ .

El algoritmo p-LMS: El algoritmo p-LMS funciona realizando un descenso más pronunciado aproximado sobre la superficie de MSE en el espacio de pesos. Debido a que es una función cuadrática de los pesos, esta superficie es convexa y posee un único mínimo (global). Un gradiente instantáneo basado en el cuadrado del error lineal instantáneo es

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (30)$$

El LMS funciona utilizando esta estimación burda del gradiente en lugar del gradiente verdadero  $\nabla J(\mathbf{W})$  de la Ec. (28). Al realizar esta sustitución en la Ec. (21) se obtiene

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (31)$$

El gradiente instantáneo se utiliza porque está fácilmente disponible a partir de una sola muestra de datos. El gradiente verdadero generalmente es difícil de obtener. El cálculo implicaría promediar los gradientes instantáneos asociados con todos los pares de entrada y salida del conjunto de entrenamiento. Esto suele ser poco práctico y casi siempre introduce un ruido de gradiente propagado en los pesos. A continuación se mostrará que p-LMS tiene la ventaja de que siempre convergerá en la media a la solución de mínimo MSE, mientras que a-LMS puede converger a una solución algo sesgada.

Realizando la diferenciación en la Ec. (31) y reemplazando el error lineal por la definición (11) se obtiene

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (32)$$

Observando que  $d(n)$  y  $\mathbf{X}(n)$  son independientes de  $\mathbf{W}(n)$ , se obtiene

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (33)$$

Este es el algoritmo p-LMS. La constante de aprendizaje  $\mu$  determina la estabilidad y la tasa de convergencia. Para patrones de entrada independientes en el tiempo, se garantiza la convergencia de la media y la varianza del vector de pesos [30] para la mayoría de los fines prácticos si

$$0 < \mu < \frac{2}{\text{trace}[\mathbf{R}]} \quad (34)$$

donde la traza  $\text{trace}[\mathbf{R}]$  (elementos diagonales de  $\mathbf{R}$ ) es la potencia promedio de la señal de los vectores  $\mathbf{X}$ , es decir,  $E(\mathbf{X}^T \mathbf{X})$ . Con esta restricción dentro de este rango, el algoritmo p-LMS converge en el

Si la matriz de autocorrelación del conjunto de vectores de patrones tiene valores propios nulos, la solución de error cuadrático medio (MSE) mínimo será un subespacio de dimensión  $m$  en el espacio de pesos [30].

Horowitz y Senne [121] han demostrado que (34) no es suficiente, en general, para garantizar la convergencia de la varianza del vector de pesos. Para patrones de entrada generados por un proceso Gaussiano de media cero e independiente en el tiempo, puede ocurrir inestabilidad en el peor caso si  $\mu$  es mayor que  $1/3 \text{ trace}[\mathbf{R}]$ .

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (35)$$

donde  $E\{\cdot\}$  denota la función de error cuadrático medio (MSE). Por lo tanto, en promedio, los pesos siguen el gradiente verdadero. En [30] se demuestra que el gradiente instantáneo es una estimación insesgada del gradiente verdadero.

Comparación de p-LMS y a-LMS: Hemos presentado ahora dos formas del algoritmo LMS, a-LMS (10) en la Sección 11V-A y p-LMS (33) en la última sección. Se trata de algoritmos muy similares, ambos utilizando el gradiente instantáneo LMS. a-LMS es auto-normalizante, con el parámetro  $\mu$  determinando la fracción del error instantáneo a corregir en cada adaptación. p-LMS es un algoritmo lineal de coeficiente constante que es considerablemente más fácil de analizar que a-LMS. Comparando ambos, el algoritmo a-LMS es similar al algoritmo p-LMS con una constante de aprendizaje continuamente variable. Aunque a-LMS es algo más difícil de implementar y analizar, se ha demostrado experimentalmente que es un mejor algoritmo que p-LMS cuando los valores propios de la matriz de autocorrelación de entrada  $\mathbf{R}$  son altamente dispares, proporcionando una convergencia más rápida para un nivel dado de ruido de gradiente propagado en los pesos. A continuación se mostrará que a-LMS tiene la ventaja de que siempre convergerá en la media a la solución de mínimo MSE, mientras que a-LMS puede converger a una solución algo sesgada.

Comenzamos con el LMS de la Ec. (10):

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu e_n \mathbf{X}_n \quad (36)$$

Reemplazando el error por su definición (11) y reordenando términos se obtiene

$$\mathbf{W}_n = \mathbf{W}_{n-1} + \mu (d(n) - \mathbf{W}_{n-1}^T \mathbf{X}_n) \mathbf{X}_n \quad (37)$$

Definimos un nuevo conjunto de entrenamiento de vectores de patrón y deseado

respuestas  $\{X_n, d_n\}$  normalizando los elementos del conjunto de entrenamiento original de la siguiente manera:

$$\mathbf{X}_n = \frac{\mathbf{X}_n}{\|\mathbf{X}_n\|} \quad (39)$$

El ruido de gradiente es la diferencia entre la estimación del gradiente y el gradiente verdadero.

La idea de un conjunto de entrenamiento normalizado fue sugerida por Derrick Nguyen.

Eq. (38) entonces se convierte en

$$W_{ii} = W_y + \alpha d, \quad \text{WIEXDX}, \quad (40)$$

Esta es la regla-LMS de la Ec. (33) con  $\alpha$  reemplazado por  $\alpha d$ . Las adaptaciones de pesos seleccionadas por la regla-LMS son equivalentes a las del algoritmo LMS presentado con un conjunto de entrenamiento diferente: el conjunto de entrenamiento normalizado definido por (39). La solución que alcanzará el algoritmo LMS es la solución de Wiener de este conjunto de entrenamiento.

$$P_s = (R)^{-1} P \quad (41)$$

Parece que no has incluido el texto fuente que debe ser traducido. Por favor, proporciona el texto completo y lo traduciré al español académico f

$$R = E[X, X] \quad (42)$$

es la matriz de correlación de entrada del conjunto de entrenamiento normalizado, y el vector

$$P = E[d, X] \quad (43)$$

es la correlación cruzada entre la entrada normalizada y la respuesta deseada normalizada. Por lo tanto,  $\alpha$ -LMS converge en la media a la solución de Wiener del conjunto de entrenamiento normalizado. Cuando los vectores de entrada son binarios con componentes  $\pm 1$ , todos los vectores de entrada tienen la misma magnitud y los dos algoritmos son equivalentes. Sin embargo, para patrones de entrenamiento no binarios, la solución de Wiener del conjunto de entrenamiento normalizado generalmente ya no es igual a la del problema original, por lo que  $\alpha$ -LMS converge en la media a una versión ligeramente sesgada de la solución óptima de mínimos cuadrados.

Lo siento, pero no hay texto fuente proporcionado después de "fined as". Por favor, proporciona el texto completo y lo traduciré al español académico f

$$\frac{d}{dt} \log \left( \frac{1}{1 + e^{-x}} \right) = \frac{x}{1 + e^{-x}} \quad (46)$$

Retropropagación para el Adaline Sigmoide: Nuestro objetivo es minimizar  $E(\cdot)$ , promediado sobre el conjunto de patrones de entrenamiento, mediante la selección adecuada del vector de pesos. Para lograr esto, derivamos un algoritmo de retropropagación para el elemento Adaline sigmoide. Se obtiene un gradiente instantáneo con cada presentación del vector de entrada, y se emplea el método de descenso más pronunciado para minimizar el error, tal como se realizó con el algoritmo z-LMS de la Ec. (33).

La idea de un conjunto de entrenamiento normalizado también puede obtenerse durante la presentación del  $k$ -ésimo vector de entrada  $X$ , está dado por

$$V_k = \frac{1}{N} \sum_{i=1}^N \frac{O_i}{1 + e^{-x_i}} \quad \text{Claro, aquí tienes l}$$

La idea de un conjunto de entrenamiento normalizado también puede utilizarse para relacionar los rangos estables de las constantes de aprendizaje  $\alpha$  y  $\mu$  en los dos algoritmos. El rango estable para  $\alpha$  en el algoritmo  $\alpha$ -LMS dado en la Ec. (15) puede calcularse a partir del rango correspondiente para  $\mu$  dado en la Ec. (34), reemplazando  $\alpha$  y  $\mu$  en la Ec. (34) por  $\alpha$  y  $\alpha/2$ , respectivamente, y observando que la traza de  $R$  es igual a uno.

Diferenciando la Ec. (46) se obtiene

$$\frac{d}{dt} \log \left( \frac{1}{1 + e^{-x}} \right) = \frac{x}{1 + e^{-x}} \quad \text{Lo siento, pero no}$$

$$0 < \alpha < 2 \quad \text{trace}[R]$$

$$0 < \alpha < 2 \quad (44)$$

Puede observarse que

$$5 = XLW \quad (49)$$

## B. Reglas No Lineales

Parece que no has incluido el texto fuente que debe ser traducido. Proporcióname el texto completo y lo traduciré al español académico f

Los elementos Adaline considerados hasta ahora utilizan en sus salidas cuantizadores de limitación dura (signos), o ninguna no linealidad en absoluto. La relación entrada-salida del cuantizador de limitación dura es  $y = \text{sgn}(x)$ . Otras formas de no linealidad han entrado en uso en las últimas dos décadas, principalmente del tipo sigmoidal. Estas no linealidades proporcionan saturación para la toma de decisiones, pero poseen características de entrada-salida diferenciables que facilitan la adaptabilidad. Generalizamos la definición del elemento Adaline para incluir el posible uso de una sigmoide en lugar del signo, y luego determinamos algoritmos de adaptación adecuados.

Sustituyendo en la Ec. (48) se obtiene

$$\frac{d}{dt} \log \left( \frac{1}{1 + e^{-x}} \right) = \frac{x}{1 + e^{-x}} \quad (51)$$

Insertando esto en la Ec. (47) se obtiene

$$V = \frac{1}{2} \log \left( \frac{1}{1 + e^{-x}} \right) \quad (52)$$

Utilizando esta estimación del gradiente junto con el método de descenso más pronunciado, se proporciona un medio para minimizar el error cuadrático medio incluso después de que la señal sumada  $\sum s_i$  atraviese la sigmoide no lineal  $\text{sgm}(S)$ . Una función sigmoide típica es la tangente hiperbólica:

Fig. 18 muestra un elemento Adaline sigmoide que incorpora una no linealidad sigmoidal. La relación entrada-salida de la sigmoide puede denotarse mediante  $y = \text{tanh}(x)$ . El algoritmo es

$$Y = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}} \quad (45)$$

Adaptaremos este Adaline con el objetivo de minimizar el error cuadrático medio de la función sigmoide  $\text{tanh}(s)$ , de-

$$W_{ir} = W_e + w(\cdot) \epsilon \quad (53)$$

$$= W + 2 \log \left( \frac{1}{1 + e^{-x}} \right) \quad (54)$$

Algoritmo (54) es el algoritmo de retropropagación para el elemento Adaline sigmoide. El nombre de retropropagación tiene más sentido cuando el algoritmo se utiliza en una capa

A continuación se presenta la t



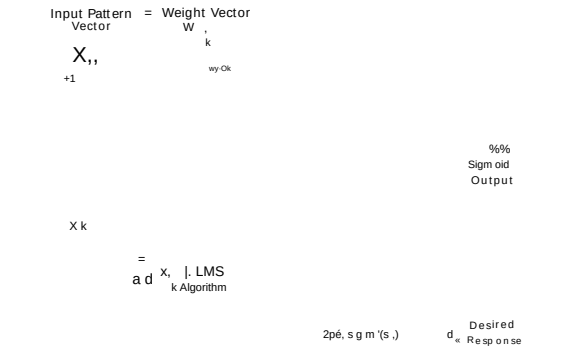


Fig. 19. Implementación de la retropropagación para el elemento Adaline sigmoide.

red neuronal, la cual se estudiará a continuación. La implementación del algoritmo (54) se ilustra en la Fig. 19.

Si la sigmoide se elige como la función tangente hiperbólica (45), entonces la derivada  $\text{sgm}'(s_k)$  viene dada por

$$\text{sgm}'(s_k) = \frac{1}{1 + \exp(-2s_k)} = 1 \cdot \tanh(s_k) = 1 \cdot y_k \quad (55)$$

De acuerdo, la Ec. (54) se convierte en

$$W_{k+1} = W_k + 2\eta e_k y_k \quad (56)$$

**\*\*Regla Madaline para la Adaline Sigmoide:\*\*** La implementación del algoritmo (54) (Fig. 19) requiere una realización precisa de la función sigmoide y su función derivada. Es posible que estas funciones no se realicen con precisión cuando se implementan con hardware analógico. De hecho, en una red analógica, cada Adaline tendrá sus propias no linealidades individuales. En la práctica, se han encontrado dificultades en la adaptación con el algoritmo de retropropagación debido a imperfecciones en las funciones no lineales.

Para sortear estos problemas, David Andes ha diseñado un nuevo algoritmo para adaptar redes de Adalines sigmoides. Este es el algoritmo de la Regla Madaline II (MRII).

La idea de MRII para un Adaline sigmoide se ilustra en la Fig. 20. La derivada de la función sigmoide no se utiliza aquí. En su lugar, se añade una pequeña señal de perturbación  $\Delta s_k$  a la suma  $s_k$ , y se observa el efecto de esta perturbación sobre la salida  $y_k$  y el error  $e_k$ .

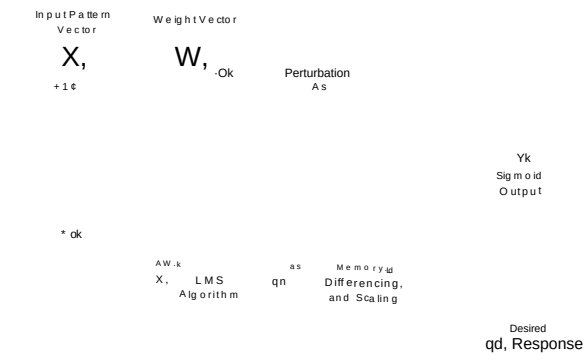


Fig. 20. Implementación del algoritmo MRII para el elemento Adaline con función sigmoide.

Una estimación instantánea del gradiente puede obtenerse de la siguiente manera:

$$\frac{\partial e_k}{\partial W_k} = \frac{\partial e_k}{\partial s_k} \frac{\partial s_k}{\partial W_k} = \frac{\partial e_k}{\partial s_k} \frac{\partial (W_k X_k)}{\partial W_k} = \frac{\partial e_k}{\partial s_k} X_k \quad (57)$$

Dado que  $\Delta s_k$  es pequeño,

$$\Delta s_k \approx \Delta s_k$$

Otra forma de obtener un gradiente instantáneo aproximado midiendo los efectos de la perturbación  $\Delta s_k$  puede obtenerse a partir de la Ec. (57).

$$\frac{\partial e_k}{\partial W_k} \approx \frac{\partial e_k}{\partial s_k} \frac{\partial s_k}{\partial W_k} = \frac{\partial e_k}{\partial s_k} X_k \quad (58)$$

De acuerdo con lo anterior, existen dos formas del algoritmo MRII para el Adaline sigmoide. Ambas se basan en el método del descenso más pronunciado, utilizando los gradientes instantáneos estimados:

$$W_{k+1} = W_k + \eta \frac{\partial e_k}{\partial W_k} X_k \quad (60)$$

o r,

$$W_{k+1} = W_k + \eta \frac{\partial e_k}{\partial s_k} X_k \quad (61)$$

Para perturbaciones pequeñas, estas dos formas son esencialmente idénticas. Ninguna de ellas requiere conocimiento a priori de la derivada de la sigmoide, y ambas son robustas frente a variaciones naturales, sesgos y deriva en el hardware analógico. La elección de la forma a utilizar es una cuestión de conveniencia de implementación. El algoritmo de la Ec. (60) se ilustra en la Fig. 20.

En relación con el algoritmo (61), pueden realizarse algunas modificaciones para establecer un punto de interés. Nótese que, de acuerdo con la Ec. (46),

Añadir la perturbación  $\Delta s_k$  provoca un cambio en  $e_k$ , igual a

Ahora, la Ec. (61) puede reescribirse como

$$W_{k+1} = W_k + \eta \frac{\partial e_k}{\partial s_k} X_k \quad (64)$$

Dado que  $\Delta s_k$  es pequeño, la relación de incrementos puede ser reemplazada por una relación de diferenciales, obteniendo finalmente

$$W_{k+1} = W_k + \eta \frac{\partial e_k}{\partial s_k} X_k \quad (65)$$

$$= W_k + 2\eta e_k \text{sgm}'(s_k) X_k \quad (66)$$

Esto es idéntico al algoritmo de retropropagación (54) para el Adaline sigmoide. Por lo tanto, la retropropagación y MRII son matemáticamente equivalentes si la perturbación  $\Delta s_k$  es pequeña, pero MRII es robusto, incluso con implementaciones analógicas.

Aquí está la traducción al español académico formal, siguiendo todas las reglas especificadas en las Superficies de Error Cuadrático Medio (MSE) del Adaline: \*\* La Fig. 21 muestra un combinador lineal conectado tanto a dispositivos sigmoide como signo. En esta figura se designan tres errores:  $e$ ,  $\hat{e}$  y  $\tilde{e}$ . Estos son:

$$\begin{aligned} \text{sigmoid error } e &= d - \text{sgm}(s) \\ \text{signum error } \hat{e} &= d - \text{sgn}(\text{sgm}(s)) \\ \text{linear error } \tilde{e} &= d - \text{sgn}(s). \end{aligned} \quad (67)$$

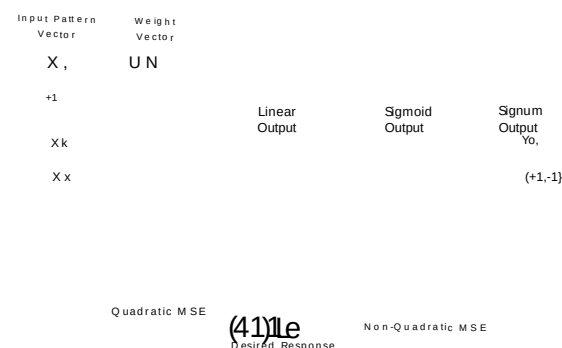


Fig. 21. Los errores lineal, sigmoide y signo de la Adaline.

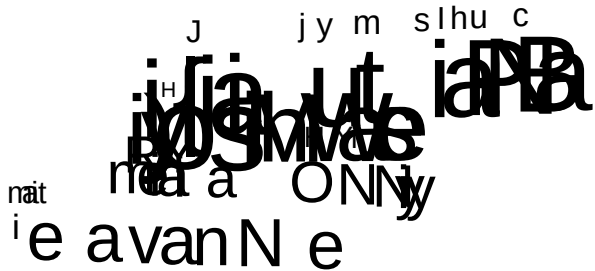


Fig. 24. Ejemplo de superficie de error cuadrático medio (MSE) de error signum.

Para demostrar la naturaleza de las superficies de error cuadrático asociadas con estos tres tipos de error, se realizó un experimento simple con un Adaline de dos entradas. El Adaline fue alimentado por un conjunto típico de patrones de entrada y sus respuestas deseadas binarias asociadas. La función de activación sigmoide utilizada fue la tangente hiperbólica. Los pesos podrían haberse adaptado para minimizar el error cuadrático medio de  $\epsilon$  (epsilon),  $\epsilon^2$  (epsilon al cuadrado) o  $\epsilon^2$  (epsilon al cuadrado). Las superficies de error cuadrático medio (MSE) de  $\epsilon$  ( $\epsilon$ ),  $\epsilon^2$  ( $\epsilon^2$ ) y  $\epsilon^2$  ( $\epsilon^2$ ), representadas en función de los dos valores de peso, se muestran en las Figs. 22 y 23, respectivamente.

que minimizar el error cuadrático medio del error de signo. Solo se garantiza que el error lineal tenga una superficie de MSE con un mínimo global único (asumiendo una matriz  $R$  invertible). Las otras superficies de MSE pueden presentar óptimos locales [122], [123

En las redes neuronales no lineales, los métodos de gradiente generalmente funcionan mejor con no linealidades sigmoideas que con las de signo. Las no linealidades sigmoideas son requeridas por las técnicas MR III y de retropropagación. Además, las redes sigmoideas son capaces de formar representaciones internas más complejas que los códigos binarios simples y, por lo tanto, estas redes pueden a menudo formar regiones de decisión más sofisticadas que aquellas asociadas con redes de signo similares. De hecho, si se pudiera construir un Adaline sigmoideo de precisión infinita y sin ruido, sería capaz de transmitir una cantidad infinita de información en cada paso de tiempo. Esto contrasta con la capacidad máxima de información de Shannon de un bit asociada con cada elemento binario.

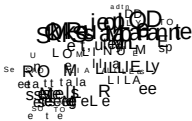


Fig. 22. Ejemplo de superficie de error cuadrático medio (MSE) de error lineal.

La función signo presenta ciertas ventajas sobre la sigmoide, ya que es más fácil de implementar en hardware y mucho más simple de calcular en una computadora digital. Además, las salidas de las funciones signo son señales binarias que pueden ser manipuladas eficientemente por computadoras digitales. En una red de funciones signo con entradas binarias, por ejemplo, la salida de cada combinador lineal puede calcularse sin realizar multiplicaciones de pesos. Esto implica simplemente sumar los valores de los pesos con entradas y restar de esta suma los valores de todos los pesos que están conectados a entradas.

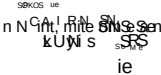


Fig. 23. Ejemplo de superficie de error cuadrático medio (MSE) de error sigmoide.

Aunque el experimento anterior no es exhaustivo, podemos inferir del mismo que minimizar el error cuadrático medio del error lineal es sencillo, y minimizar el error cuadrático medio del error sigmoide es más difícil, pero típicamente mucho más fácil.

A veces se utiliza una función signo en un Adaline para producir decisiones de salida determinantes. La probabilidad de error es entonces proporcional al error cuadrático medio de la salida  $\epsilon$ . Para minimizar aproximadamente esta probabilidad de error, se puede minimizar fácilmente  $E[\epsilon^2]$  en lugar de minimizar directamente  $E[\epsilon]$  [58]. Sin embargo, con solo un poco más de cómputo se podría minimizar  $E[\epsilon^2]$  y, por lo general, aproximarse mucho más al objetivo de minimizar  $E[\epsilon]$ . Por lo tanto, la sigmoide puede utilizarse en el entrenamiento de los pesos incluso cuando se emplea la función signo para formar la salida del Adaline, como se muestra en la Fig. 21.

#### VII. REGLAS DE DESCENSO MÁS PRONUNCIADO REDES MULTIELEMENTO

Ahora estudiamos reglas para la minimización mediante descenso más pronunciado del error cuadrático medio (MSE) asociado con redes completas de elementos Adaline sigmoideos. Al igual que sus contrapartes de un solo elemento, las reglas de descenso más pronunciado más prácticas y eficientes para redes multielemento suelen trabajar con una presentación de patrón a la vez. Describiremos dos reglas de descenso más pronunciado para redes sigmoideas multielemento: la retropropagación y la Regla Madaline III.

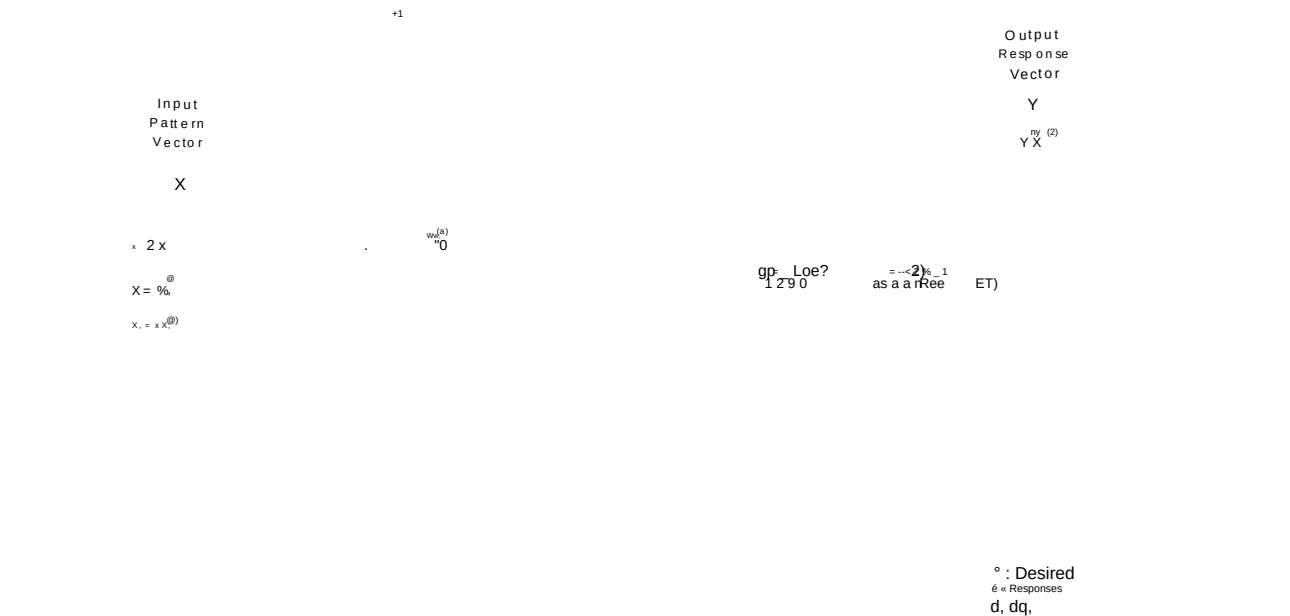


Fig. 25. Ejemplo de arquitectura de red de retropropagación de dos capas.

#### A. Retropropagación para Redes

La publicación de la técnica de retropropagación por Rumelhart et al. [42] ha sido, sin duda, el desarrollo más influyente en el campo de las redes neuronales durante la última década. En retrospectiva, la técnica parece simple. No obstante, en gran medida debido a que las primeras investigaciones en redes neuronales se ocupaban casi exclusivamente de no linealidades de limitación dura, la idea nunca se les ocurrió a los investigadores de redes neuronales durante la década de 1960.

Los conceptos básicos de la retropropagación se comprenden fácilmente. Desafortunadamente, estas ideas simples suelen verse oscurecidas por una notación relativamente intrincada, por lo que las derivaciones formales de la regla de retropropagación resultan a menudo tediosas. Presentamos una derivación informal del algoritmo e ilustramos su funcionamiento para la red simple mostrada en la Fig. 25.

La técnica de retropropagación constituye una generalización no trivial del caso de Adaline sigmoide monocapa de la Sección VI-B. Cuando se aplica a redes de múltiples elementos, la técnica de retropropagación ajusta los pesos en la dirección opuesta al gradiente de error instantáneo:

$$\Delta w_{kj} = -\frac{\partial e}{\partial w_{kj}} = -\frac{\partial}{\partial w_{kj}} \left( \frac{1}{2} \sum_{k=1}^K (d_k - y_k)^2 \right) \quad (68)$$

Ahora, sin embargo,  $\mathbf{W}$  es un vector largo de  $m$  componentes que contiene todos los pesos de la red completa. El error cuadrático instantáneo  $e_j$  es la suma de los cuadrados de los errores en cada una de las  $N$  salidas de la red. Por lo tanto,

$$e = \sum_{j=1}^N e_j \quad (69)$$

En el ejemplo de red mostrado en la Fig. 25, el error cuadrático total se expresa mediante

$$e = (d_1 - y_1)^2 + (d_2 - y_2)^2 \quad (70)$$

donde ahora suprimimos el índice temporal  $k$  por conveniencia. En su forma más simple, el entrenamiento por retropropagación comienza presentando un vector de patrón de entrada  $X$  a la red, avanzando hacia adelante a través del sistema para generar un vector de respuesta de salida  $Y$ , y calculando los errores en cada salida. El siguiente paso implica propagar los efectos de los errores hacia atrás a través de la red para asociar una 'derivada del error cuadrático'  $\delta$  con cada Adaline, calculando un gradiente a partir de cada  $\delta$ , y finalmente actualizando los pesos de cada Adaline basándose en el gradiente correspondiente. Luego se presenta un nuevo patrón y el proceso se repite. Los valores iniciales de los pesos normalmente se establecen en números aleatorios pequeños. El algoritmo no funcionará correctamente con redes multicapa si los pesos iniciales son cero o valores no nulos mal elegidos.

Podemos obtener una idea de lo que implica el cálculo asociado al algoritmo de retropropagación examinando la red de la Fig. 25. Cada uno de los cinco círculos grandes representa un combinador lineal, así como algunas trayectorias de señal asociadas para la retropropagación del error y la maquinaria adaptativa correspondiente para la actualización de los pesos. Este detalle se muestra en la Fig. 26. Las líneas continuas en estos diagramas representan las trayectorias de señal hacia adelante a través de la red.

Recientemente, Nguyen ha descubierto que una selección más sofisticada de los valores iniciales de los pesos en las capas ocultas puede reducir los problemas con los óptimos locales y aumentar drásticamente la velocidad de entrenamiento de las redes [100]. La evidencia experimental sugiere que es recomendable elegir los pesos iniciales de cada capa oculta de manera cuasi aleatoria, lo que garantiza que, en cada posición del espacio de entrada de una capa, las salidas de todas sus Adalines, excepto unas pocas, estén saturadas, mientras que se asegura que cada Adaline en la capa no esté saturada en alguna región de su espacio de entrada. Cuando se utiliza este método, los pesos en la capa de salida se establecen en valores aleatorios pequeños.

Observamos que el segundo término es cero. En consecuencia,

$$\delta_j = -\frac{\partial E}{\partial s_j} = -\frac{\partial}{\partial s_j} \left( \frac{1}{2} \sum_k (s_k - t_k)^2 \right) \quad (75)$$

Observando que  $\delta_j$  y  $\delta_{s_j}$  son independientes se obtiene

$$\delta_j = -\frac{\partial E}{\partial s_j} = -\frac{\partial}{\partial s_j} \left( \frac{1}{2} \sum_k (s_k - t_k)^2 \right) = -\sum_k (s_k - t_k) \frac{\partial s_k}{\partial s_j} \quad (76)$$

$$= -\sum_k (s_k - t_k) \delta_{s_j} \quad (77)$$

We denote the error  $\delta_j$  as  $\delta_j$ . Therefore,

$$\delta_j = -\sum_k (s_k - t_k) \delta_{s_j} \quad (78)$$

Note that this corresponds to the calculation of  $\delta_j$ , as shown in Fig. 25. The value of  $\delta_j$  associated with the other output element in the figure can be expressed in an analogous fashion. Thus each square-error derivative  $\delta_j$  in the output layer is computed by summing the output error

$$\delta_j = -\sum_k (s_k - t_k) \delta_{s_j} \quad (79)$$

Fig. 26. Detalle del combinador lineal y circuitos asociados en la red de retropropagación.

and the dotted lines represent the inverse trajectories separated that are used in association with the calculations of the derivatives of the square error [5]. In Fig. 25, observe that the calculations associated with the backward pass have a complexity approximately equal to the represented by the forward pass through the network. The backward pass requires the same number of calculations of function that the forward pass, but it implies multiplications of weights in the first layer.

Figure 25. The value of  $\delta_j$  associated with the other

output element in the figure can be expressed in an anal-

ogous fashion. Thus each square-error derivative  $\delta_j$  in the

output layer is computed by summing the output error

Como se indicó anteriormente, una vez que se ha presentado un patrón a la red y se ha calculado el error de respuesta de cada salida, el siguiente paso del algoritmo de retropropagación implica encontrar la derivada instantánea del error cuadrático  $\delta_j$  asociada con cada unión de suma en la red. La derivada de la capa  $j$  del error cuadrático asociada con el  $j$ -ésimo Adaline en la capa  $j$  se define como:

Desarrollar expresiones para las derivadas del error cuadrático asociadas con las capas ocultas no es mucho más difícil (consultar la Fig. 25). Necesitamos una expresión para  $\delta_j$ , el error cuadrático de la capa  $j$ . La derivada  $\delta_j$  se define mediante

$$\delta_j = -\frac{\partial E}{\partial s_j} \quad (80)$$

Desarrollando esto mediante la regla de la cadena, y observando que  $\delta_j$  está determinado enteramente por los valores de  $\delta_{s_j}$  y  $\delta_{t_j}$ , se obtiene

Utilizando las definiciones de  $\delta_j$  y  $\delta_{s_j}$ , y sustituyendo posteriormente las versiones expandidas de las salidas lineales de Adaline  $s_j$  y  $t_j$ , obtenemos

Each  $\delta_j$  (Fig. 25) tienes la

Cada una de estas derivadas nos indica, en esencia, qué tan sensible es el error de salida a de suma cuadrática de la red a los cambios en la salida lineal del elemento Adaline asociado.

Las derivadas del error cuadrático instantáneo se calculan primero para cada elemento en la capa de salida. El cálculo es simple. Como ejemplo, a continuación derivamos la expresión requerida para  $\delta_j$ , la derivada asociada con el elemento Adaline superior en la capa de salida de la Fig. 25. Comenzamos con la definición de  $\delta_j$  a partir de la Ec. (71).

$$\delta_j = -\frac{\partial E}{\partial s_j} \quad (81)$$

Al expandir el término de error cuadrático  $E$  mediante la Ec. (70) se obtiene

$$E = \frac{1}{2} \sum_j (s_j - t_j)^2 = \frac{1}{2} \sum_j (y_j - t_j)^2 \quad (82)$$

$$= \frac{1}{2} \sum_j (y_j - t_j)^2 = \frac{1}{2} \sum_j (y_j - t_j)^2 \quad (83)$$

$$= \frac{1}{2} \sum_j (y_j - t_j)^2 = \frac{1}{2} \sum_j (y_j - t_j)^2 \quad (84)$$

Notando que  $\frac{\partial}{\partial s_j} \left( \sum_k (s_k - t_k)^2 \right) = 2(s_j - t_j)$ , para  $j = 1, 2, \dots, N$

$$\delta_j = -\frac{\partial E}{\partial s_j} = -(s_j - t_j) \quad (85)$$

Ahora, realizamos la siguiente definición

$$\delta_j = -\frac{\partial E}{\partial s_j} = -(s_j - t_j) \quad (86)$$

De acuerdo,

$$\delta_j = -\frac{\partial E}{\partial s_j} = -(s_j - t_j) \quad (87)$$

Haciendo referencia a la Fig. 25, podemos recorrer el circuito para verificar que  $\delta_j$  se calcula de acuerdo con las Ecs. (86) y (87).

La forma más sencilla de encontrar los valores de  $\delta_k$  para todos los elementos Adaline en la red es seguir el diagrama esquemático de la Fig. 25.

Por lo tanto, el procedimiento para encontrar  $\delta_k$ , la derivada del error cuadrático asociada con un Adaline dado en la capa oculta  $\delta_k$ , implica multiplicar respectivamente cada derivada  $\delta_{k+1}$  asociada con cada elemento en la capa inmediatamente posterior a un Adaline dado por el peso que lo conecta con dicho Adaline. Estas derivadas del error cuadrático ponderadas se suman entonces, produciendo un término de error  $\delta_k$ , el cual, a su vez, se multiplica por  $\text{sgm}'(s_k)$ , la derivada de la función de activación sigmoide evaluada en  $s_k$ . Este proceso de retropropagación de las derivadas instantáneas del error cuadrático desde una capa hasta la capa inmediatamente precedente se repite sucesivamente hasta obtener una derivada del error cuadrático  $\delta_k$  en cada capa, repitiendo el argumento de la regla de la cadena asociado con la Ec. (81).

Ahora disponemos de un método general para encontrar una derivada para cada elemento Adaline en la red. El siguiente paso es utilizar estas derivadas para obtener los gradientes correspondientes. Considérese una Adaline en algún lugar de la red, que, durante la presentación  $k$ , tiene un vector de pesos  $\mathbf{w}$  y un vector de entrada  $\mathbf{x}$  y una salida lineal  $\hat{y} = \mathbf{w} \cdot \mathbf{x}$ .

El gradiente instantáneo para este elemento Adaline es

$$\bar{y} = \frac{dE}{d\hat{y}} \quad (88)$$

Esto puede escribirse como

$$\bar{y} = \frac{dE}{d\hat{y}} = \frac{dE}{ds} \frac{ds}{d\hat{y}} = \frac{dE}{ds} \frac{1}{\text{sgm}'(s)} \quad (89)$$

Nota que  $\delta_k$  y  $\delta_{k+1}$  son independientes, por lo tanto

$$\delta_k = \frac{dE}{d\hat{y}} = \frac{dE}{ds} \frac{ds}{d\hat{y}} = \frac{dE}{ds} \frac{1}{\text{sgm}'(s)} \quad (90)$$

Parece que no has incluido el texto fuente que debe ser traducido. Proporcióname el texto completo y lo traduciré al español.

$$\delta_k = \frac{dE}{d\hat{y}} = \frac{dE}{ds} \frac{ds}{d\hat{y}} = \frac{dE}{ds} \frac{1}{\text{sgm}'(s)} \quad (91)$$

Para este elemento,

$$\delta_k = \frac{1}{2} \frac{dE}{ds} \frac{ds}{d\hat{y}} = \frac{1}{2} \frac{dE}{ds} \frac{1}{\text{sgm}'(s)} \quad (92)$$

De acuerdo,

$$\delta_k = \frac{1}{2} \frac{dE}{ds} \frac{ds}{d\hat{y}} = \frac{1}{2} \frac{dE}{ds} \frac{1}{\text{sgm}'(s)} \quad (93)$$

Actualizando los pesos del elemento Adaline mediante el método de descenso más pronunciado con el gradiente instantáneo es un proceso representado por

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \eta \delta_k \mathbf{x} \quad (94)$$

Así, tras retropropagar todas las derivadas del error cuadrático, completamos una iteración de retropropagación añadiendo a cada vector de pesos el vector de entrada correspondiente escalado por la derivada del error cuadrático asociada. La Ec. (94) y los medios para encontrar  $\delta_k$  constituyen la regla general de actualización de pesos del algoritmo de retropropagación.

Existe una gran similitud entre la ecuación (94) y el algoritmo p-LMS (33), pero esta similitud debe considerarse con cautela. La cantidad  $\delta_k$ , definida como una derivada del error cuadrático,

podría parecer que desempeña el mismo papel en la retropropagación que el que juega el error en el algoritmo u-LMS. Sin embargo, no es un error. La adaptación del Adaline dado se efectúa para reducir el error cuadrático de salida  $e^2$ , no  $e$ , de dicho Adaline o de cualquier otro Adaline en la red. El objetivo no es reducir los de la red, sino reducir  $e$  en la salida de la red.

Es interesante examinar las actualizaciones de pesos que la retropropagación impone sobre los elementos Adaline en la capa de salida. Sustituyendo la Ec. (77) en la Ec. (94) se revela que el Adaline que proporciona la salida  $\hat{y}$ , en la Fig. 25, se actualiza mediante la regla

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \eta \delta_k \mathbf{x} \quad (95)$$

Esta regla resulta ser idéntica a la versión de Adaline individual (54) de la regla de retropropagación. Esto no es sorprendente, ya que el Adaline de salida recibe tanto las señales de entrada como las respuestas deseadas, por lo que su circunstancia de entrenamiento es la misma que la experimentada por un Adaline entrenado de forma aislada.

Existen muchas variantes del algoritmo de retropropagación. En ocasiones, el tamaño de  $\delta_k$  se reduce durante el entrenamiento para disminuir los efectos del ruido de gradiente en los pesos. Otra extensión es la técnica de momento [42], que consiste en incluir en el vector de cambio de pesos  $\Delta \mathbf{w}$  de cada Adaline un término proporcional al cambio de peso correspondiente de la iteración anterior. Es decir, la Ec. (94) se reemplaza por un par de ecuaciones:

donde la constante de momento  $0 \leq \alpha \leq 1$  en la práctica se establece usualmente en un valor cercano a 0.8 o 0.9.

La técnica de momento filtra pasa-bajos las actualizaciones de los pesos y, por lo tanto, tiende a resistir cambios erráticos en los pesos causados ya sea por ruido de gradiente o por altas frecuencias espaciales en la superficie de error cuadrático medio (MSE). El factor  $(1 - \alpha)$  en la Ec. (96) se incluye para otorgar al filtro una respuesta de 0.5 unitaria de modo que la tasa de aprendizaje  $\eta$  no necesite reducirse a la medida que se incrementa la constante de momento  $\alpha$ . También se puede agregar un término de momento a las ecuaciones de actualización de los algoritmos descritos en este artículo. Un análisis detallado de los problemas de estabilidad asociados con la actualización mediante momento para el algoritmo  $\mu$ -LMS, por ejemplo, ha sido descrito por Shynk y Roy [124].

En nuestra experiencia, la técnica de momento utilizada por sí sola suele tener poco valor. Sin embargo, hemos encontrado que a menudo resulta útil aplicar esta técnica en situaciones que requieren estimaciones de gradiente relativamente "limpias". Un caso es una ecuación de actualización de pesos normalizada, que hace que el vector de pesos de la red se desplace la misma distancia euclidiana en cada iteración. Esto puede lograrse reemplazando las Ecs. (96) y (97) por

$$\mathbf{w}_{k+1} = \frac{\mathbf{w}_k + \eta \delta_k \mathbf{x}}{\|\mathbf{w}_k + \eta \delta_k \mathbf{x}\|} \quad (98)$$

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \frac{\eta \delta_k \mathbf{x}}{\|\mathbf{w}_k + \eta \delta_k \mathbf{x}\|} \quad (99)$$

Aquí está la traducción al español académico formal, siguiendo estrictamente todas las reglas especificadas nuevamente.  $0 < \alpha < 1$ . Las actualizaciones de pesos determinadas por las Ecs. (98) y (99) pueden ayudar a una red a encontrar una solución cuando se encuentra una región local relativamente plana en la superficie de MSE.

•  $C$  le a  $n$  · gradient estimates are those with little gradient noise.

Los pesos se desplazan en la misma magnitud independientemente de si la superficie es plana o inclinada. Esto recuerda al algoritmo LMS debido a que el término de gradiente en la ecuación de actualización de pesos se normaliza mediante un factor variable en el tiempo. La regla de actualización de pesos podría modificarse aún más incluyendo términos de ambas técnicas asociadas con las Ecs. (96) a (99). Otros métodos para acelerar el entrenamiento por retropropagación incluyen el popular método quickprop de Fahlman [125], así como el enfoque delta-bar-delta reportado en un excelente artículo de Jacobs [126].

Una de las áreas más prometedoras en la investigación de redes neuronales involucra variantes de retropropagación para entrenar diversas redes recurrentes (con retroalimentación de señales). Recientemente, se han derivado reglas de retropropagación para entrenar redes recurrentes con el fin de aprender asociaciones estáticas [127], [128]. Más interesante resulta la técnica en línea de Williams y Zipser [129], que permite que una amplia clase de redes recurrentes aprenda asociaciones y trayectorias dinámicas. Una variante más general y computacionalmente viable de esta técnica ha sido propuesta por Narendra y Parthasarathy [104]. Estos métodos en línea son generalizaciones de un conocido algoritmo de descenso más pronunciado para entrenar filtros IIR lineales [130], [30].

Una técnica equivalente que suele ser mucho menos intensiva computacionalmente, pero más adecuada para el cálculo fuera de línea [37], [42], [131], denominada "retropropagación a través del tiempo", ha sido utilizada por Nguyen y Widrow [50] para permitir que una red neuronal aprenda sin un maestro cómo estacionar en reversa a un camión con remolque simulado por computadora hasta un muelle de carga (Fig. 27). Esta es una tarea de dirección altamente no lineal y aún no se sabe cómo diseñar un controlador para realizarla. No obstante, con solo 6 entradas que proporcionan información sobre la posición actual del camión, una red neuronal bicapa con solo 26 Adalines fue capaz de aprender por sí misma a resolver este problema. Una vez entrenada, la red pudo estacionar en reversa el camión desde cualquier posición y orientación inicial frente al muelle de carga.

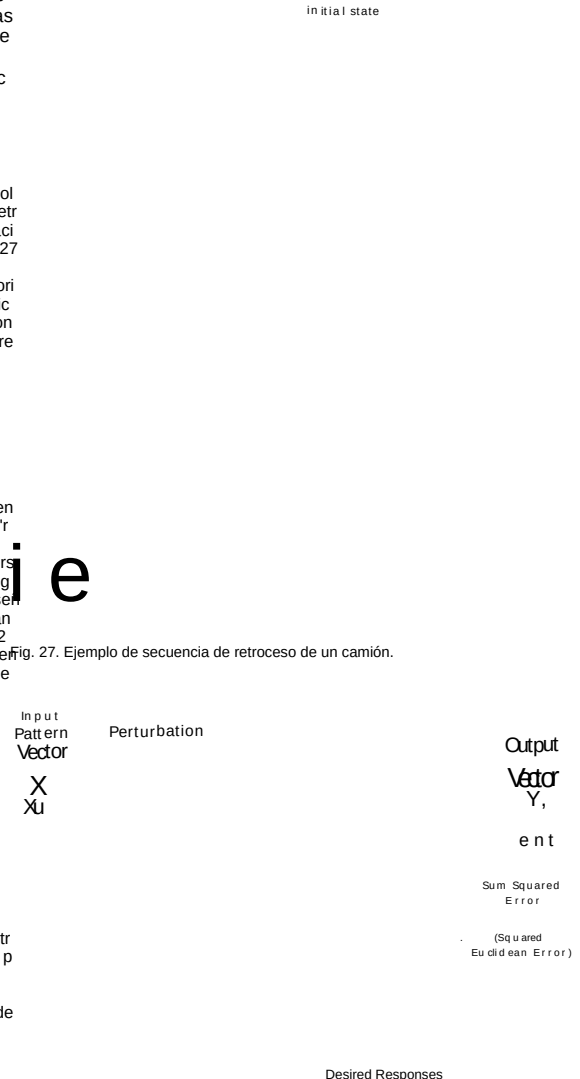


Fig. 28. Ejemplo de arquitectura Madaline II de dos capas.

## B. Regla Madaline III para Redes

Es difícil construir redes neuronales con hardware analógico que puedan ser entrenadas eficazmente mediante la popular técnica de retropropagación. Los intentos por superar esta dificultad han llevado al desarrollo del algoritmo MRIII. Un chip neurocomputacional analógico comercial basado principalmente en este algoritmo ya ha sido diseñado [132]. El método descrito en esta sección es una generalización de la técnica MRIII para un solo Adaline (60). La generalización multielemento de la otra regla MRIII para un solo elemento (61) se describe en [133].

El algoritmo MRIII puede describirse fácilmente haciendo referencia a la Fig. 28. Aunque esta figura muestra una arquitectura simple de prealimentación de dos capas, el procedimiento que se desarrollará funcionará para redes neuronales con cualquier número de Adaline, incluso aquellas con retroalimentación de señal.

El artículo de Jacobs [3], al igual que muchos otros en la literatura, supone para su análisis que se utilizan los gradientes verdaderos, en lugar de los gradientes instantáneos, para actualizar los pesos; es decir, que los pesos se modifican periódicamente, solo después de haber presentado todos los patrones de entrenamiento. Esto elimina el ruido del gradiente, pero puede ralentizar enormemente el entrenamiento si el conjunto de entrenamiento es grande. El procedimiento delta-bar-delta en el artículo de Jacobs implica monitorear los cambios de los gradientes verdaderos en respuesta a las modificaciones de los pesos. En este caso, debería ser posible evitar el costo de calcular explícitamente los gradientes verdaderos, monitoreando en su lugar los cambios en las salidas de, por ejemplo, dos filtros de momento con diferentes constantes de tiempo.

elementos en cualquier estructura de prealimentación. En [133], se analizan variantes del enfoque MRIII básico que permiten aplicar el entrenamiento por descenso más pronunciado a topologías de red más generales, incluso aquellas con retroalimentación de señal.

Asuma que un patrón de entrada  $X$  y sus respuestas de salida deseadas asociadas  $(d_1)$  y  $(d_2)$  se presentan a la red de la Fig. 28. En este punto, medimos el error de la suma de los cuadrados de las respuestas de salida  $(\epsilon^2 = (d_1 - y_1)^2 + (d_2 - y_2)^2 = \epsilon_1^2 + \epsilon_2^2)$ . Luego, añadimos una pequeña cantidad  $(\Delta s)$  a un Adaline seleccionado en la red, proporcionando una perturbación a la suma lineal del elemento. Esta perturbación se propaga a través de la red y provoca un cambio en la suma de los cuadrados de los errores,  $(\Delta(\epsilon^2) = \Delta(\epsilon_1^2 + \epsilon_2^2))$ . Una relación fácilmente medible es

$$\Delta(\epsilon^2) \approx \Delta(\epsilon_1^2 + \epsilon_2^2) \approx 2 \epsilon_1 \Delta \epsilon_1 + 2 \epsilon_2 \Delta \epsilon_2 \quad (100)$$

A continuación, utilizamos esto para obtener el gradiente instantáneo de  $\sum_k (e_k)^2$  con respecto al vector de pesos del Adaline seleccionado. Para la  $k$ -ésima presentación, el gradiente instantáneo es

$$\frac{d}{dw} \sum_k (e_k)^2 = \sum_k 2e_k \frac{de_k}{ds_k} \cdot \frac{ds_k}{dw} = -2e_k x_k \quad (101)$$

Reemplazar la derivada por un cociente de diferencias produce

$$\frac{d}{dw} \sum_k (e_k)^2 \approx \frac{\sum_k (e_k)^2 - \sum_k (e_k + \Delta w)^2}{\Delta w} = -2e_k x_k \quad (102)$$

La idea de obtener una derivada mediante la perturbación de la salida lineal del elemento Adaline seleccionado es la misma que la expresada para el elemento único en la Sección VI-B, con la excepción de que aquí el error se obtiene a partir de la salida de una red de múltiples elementos, en lugar de la salida de un elemento único.

El gradiente (102) puede utilizarse para optimizar el vector de pesos de acuerdo con el método de descenso más pronunciado:

$$w_i = w_i - \eta \sum_k A_k(e_k) x_{ki} \quad (103)$$

Manteniendo el mismo patrón de entrada, se podría perturbar secuencialmente todos los elementos de la red, adaptando después de cada cálculo de gradiente, o bien se podrían calcular y almacenar las derivadas para permitir que todos los Adalines se adapten simultáneamente. Estos dos enfoques MRIII implican la misma ecuación de actualización de pesos (103), y si  $\Delta w$  es pequeño, ambos conducen a soluciones equivalentes. Con un  $\Delta w$  grande, la experiencia indica que adaptar un elemento a la vez resulta en convergencia tras menos iteraciones, especialmente en redes grandes. Sin embargo, almacenar los gradientes tiene la ventaja de que, después de medir el error no perturbado inicial durante una presentación de entrenamiento determinada, cada estimación del gradiente requiere únicamente la medición del error perturbado. Si las adaptaciones ocurren después de cada medición de error, tanto los errores perturbados como los no perturbados deben medirse para cada cálculo de gradiente. Esto se debe a que cada actualización de peso modifica el error no perturbado asociado.

### C. Comparación de MRIII con MRII

MRIII se derivó de MRII reemplazando las no linealidades de signo por sigmoides. La similitud de estos algoritmos se hace evidente al comparar la Fig. 28, que representa MRIII, con la Fig. 16, que representa MRII.

La red MRII es altamente discontinua y no lineal. No es posible utilizar un gradiente instantáneo para ajustar los pesos. De hecho, a partir de la superficie de MSE para la Adaline con función signo presentada en la Sección VI-B, queda claro que incluso las técnicas de descenso de gradiente que emplean el gradiente verdadero podrían enfrentar problemas graves con mínimos locales. Sin embargo, la idea de añadir una perturbación a la suma lineal de un elemento Adaline seleccionado es factible. Si el error de Hamming se ha reducido mediante la perturbación, la Adaline se adapta para invertir su decisión de salida. Este cambio de peso se realiza en la dirección de LMS, a lo largo de su vector  $X$ . Si la adaptación de la Adaline no redujera el error de salida de la red, no se adapta. Esto concuerda con el principio de perturbación mínima. Las Adalines seleccionadas para una posible adaptación son aquellas cuyas sumas analógicas están más cercanas a cero, es decir, las Adalines que pueden adaptarse para dar respuestas opuestas a los cambios de peso más pequeños. Es útil señalar que, con respuestas deseadas binarias  $\pm 1$ , el error de Hamming es igual a  $1/4$  del

El algoritmo MRIII funciona de manera similar. Todos los Adalines en la red MRII se adaptan, pero aquellos cuyas sumas analógicas están más cercanas a cero generalmente se adaptarán con mayor intensidad, debido a que la sigmoide tiene su pendiente máxima en cero, lo que contribuye a valores altos de gradiente. Al igual que con MRII, el objetivo es modificar los pesos para la presentación de entrada dada, con el fin de reducir el error cuadrático medio en la salida de la red. De acuerdo con el principio de mínima perturbación, los vectores de pesos de los elementos Adaline se adaptan en la dirección LMS, a lo largo de sus  $X$ -vectores, y se adaptan en proporción a su capacidad para reducir el error cuadrático medio (el cuadrado del error euclidiano) en la salida.

### D. Comparación de MRIII con Retropropagación

En la Sección VI-B, argumentamos que, para el elemento Adaline sigmoide, el algoritmo MRIII (61) es esencialmente equivalente al algoritmo de retropropagación (54). El mismo argumento puede extenderse a la red de elementos Adaline, demostrando que si  $\Delta s$  es pequeño y la adaptación se aplica a todos los elementos de la red simultáneamente, entonces MRIII es esencialmente equivalente a la retropropagación. Es decir, en la medida en que la derivada muestral  $\sum_k (de_k/ds_k)$  de la Ec. (103) sea igual a la derivada analítica  $\sum_k (de_k/ds_k)$  de la Ec. (91), ambas reglas siguen gradientes instantáneos idénticos y, por lo tanto, realizan actualizaciones de pesos idénticas.

El algoritmo de retropropagación requiere menos operaciones que MRIII para calcular los gradientes, ya que puede aprovechar el conocimiento a priori de las no linealidades sigmoideas y sus funciones derivadas. Por el contrario, el algoritmo MRIII no utiliza conocimiento previo sobre las características de las funciones sigmoideas. En cambio, adquiere gradientes instantáneos a partir de mediciones de perturbación. Mediante el uso de MRIII, las tolerancias en las implementaciones sigmoideas pueden relajarse considerablemente en comparación con las tolerancias aceptables para una retropropagación exitosa.

El entrenamiento por descenso más pronunciado de redes multicapa implementado mediante simulación computacional o mediante hardware digital paralelo preciso suele realizarse de manera óptima mediante retropropagación. Durante cada presentación de entrenamiento, el método de retropropagación requiere únicamente un cómputo hacia adelante a través de la red, seguido de un cómputo hacia atrás, para ajustar todos los pesos de una red completa. Para lograr el mismo efecto con la forma de MRIII que actualiza todos los pesos simultáneamente, se mide el error no perturbado seguido de una cantidad de mediciones de error perturbado igual al número de elementos en la red. Esto podría requerir una gran cantidad de cómputo.

Si una red ha de implementarse en hardware analógico, sin embargo, la experiencia ha demostrado que MRIII ofrece ventajas significativas sobre la retropropagación. La comparación de la Fig. 25 con la Fig. 28 demuestra la relativa simplicidad de MRIII. Se elimina todo el aparato para la propagación hacia atrás de las señales relacionadas con el error, y los pesos no necesitan transportar señales en ambas direcciones (véase la Fig. 26). MRIII es un algoritmo mucho más simple de construir y de comprender, y en principio produce el mismo gradiente instantáneo que el algoritmo de retropropagación. La técnica de momento y la mayoría de las otras variantes comunes del algoritmo de retropropagación pueden aplicarse al entrenamiento de MRIII.

En la Sección VI-B se mostraron superficies de error cuadrático medio (MSE) típicas de Adalines sigmoideas y de signo, lo que indica que las Adalines sigmoideas son mucho más propicias para los enfoques de gradiente que las Adalines de signo. El mismo fenómeno se produce cuando las Adalines se incorporan en redes de múltiples elementos. Las superficies de MSE de las redes MR-II son razonablemente caóticas y no se explorarán aquí. En esta sección examinamos únicamente superficies de MSE provenientes de un problema típico de entrenamiento con retropropagación en una red neuronal sigmoidea.

En una red con más de dos pesos, la superficie del error cuadrático medio (MSE) es de alta dimensionalidad y difícil de visualizar. No obstante, es posible examinar secciones de esta superficie representando gráficamente la superficie del MSE generada al variar dos de los pesos mientras se mantienen constantes todos los demás. Las superficies representadas en las Figs. 29

Fig. 29. Ejemplo de superficie de MSE de una red no entrenada

work as a function of two first-layer weights.

Fig. 30. Ejemplo de superficie de error cuadrático medio (MSE) de una red sigmoidea entrenada en función de dos pesos de la primera capa.

Las figuras 29 y 30 muestran dos cortes de este tipo de la superficie de MSE de un problema de aprendizaje típico, correspondientes respectivamente a una red sigmoidea no entrenada y a una entrenada. La primera superficie resultó de variar dos pesos de la primera capa de una red no entrenada. La segunda superficie resultó de variar los mismos dos pesos después de que la red fue completamente entrenada. Ambas superficies son similares, pero la segunda presenta un mínimo más profundo, el cual fue esculpido por el proceso de aprendizaje de retropropagación. Las figuras 31 y 32 resultaron de variar un conjunto diferente de dos pesos en la misma red. La figura 31 es el resultado de variar un peso de la primera capa y un peso de la tercera capa en la red no entrenada, mientras que la figura 32 es la superficie que resultó de variar los mismos dos pesos después de que la red fue entrenada.

Fig. 31. Ejemplo de superficie de MSE de una red sigmoidea no entrenada en función de un peso de la primera capa y un peso de la tercera capa.

Fig. 32. Ejemplo de superficie de MSE de una red sigmoidea entrenada en función de un peso de la primera capa y un peso de la tercera capa.

Al estudiar numerosas gráficas, resulta evidente que la retropropagación y MR-II estarán sujetas a convergencia en óptimos locales. Lo mismo ocurre con MR-I. El remedio más común para esto es la adición esporádica de ruido a los pesos o gradientes. Algunos de los métodos de "recocido simulado" [47] realizan esto. Otro método implica reentrenar la red varias veces utilizando diferentes valores iniciales de pesos aleatorios hasta encontrar una solución satisfactoria.

Las soluciones encontradas por las personas en la vida cotidiana generalmente no son óptimas, pero muchas de ellas son útiles. Si un óptimo local produce un rendimiento satisfactorio, a menudo simplemente no es necesario buscar una solución mejor.

Este año se conmemora el 30.º aniversario de la publicación de la regla del Perceptrón por Rosenblatt y del algoritmo LMS por Widrow y Hoff. También han trascurrido 16 años desde que Werbos publicó por primera vez el algoritmo de retropropagación. Estas reglas de aprendizaje, junto con varias otras, han sido estudiadas y comparadas. Aunque difieren significativamente entre sí, todas pertenecen a la misma "familia".

Se estableció una distinción entre las reglas de corrección de error y las reglas de descenso más pronunciado. Las primeras incluyen la regla del Perceptrón, las reglas de Mays, el algoritmo  $\alpha$ -LMS, la regla original de Madalene [4962] y la regla Madaline II. Las segundas incluyen el algoritmo  $\gg$ -LMS, la regla Madaline III y la



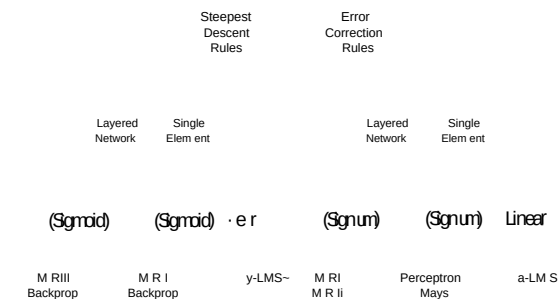


Fig. 33. Reglas de aprendizaje.

algoritmo de retropropagación. La Fig. 33 clasifica las reglas de aprendizaje que han sido estudiadas.

Aunque estos algoritmos se han presentado como reglas de aprendizaje establecidas, no debe crearse la impresión de que son perfectos e inmutables para siempre. En los posibles variaciones para cada uno de ellos. Deben considerarse como sustratos sobre los cuales construir reglas nuevas y mejores. Hay una cantidad inmensa de inventiva esperando "entre bastidores". Esperamos con interés los próximos 30 años.

A continuación, se presenta la traducción del texto proporcionado al español académico formal, siguiendo estrictamente todas las instrucciones.

- (1) Aquí está la traducción al español académico formal, siguiendo todas las reglas específicas.
- (2) K. Steinbuch y V. A. W. Piske, "Learning matrices and their applications", IEEE Trans. Electron. Comput., vol. EC-12, pp. 846-862, dic. 1963. B. Widrow, "Generalization and information storage in networks of adaptive elements", en "Self-Organizing Systems 1962", M. Yovitz, G. Jacobi, y G. Goldstein, Eds. Washington, DC: Spartan Books, 1962, pp. 435-461. L. Stark, M. Okajima, y G. H. Whitfield, "Computer pattern recognition techniques: Electrocardiographic diagnosis", IEEE Trans. Comput. Mach., vol. 5, pp. 527-532, oct. 1962. F. Rosenblatt, "Two theorems of statistical separability in the perceptron", Mechanization of Thought Processes: Proceedings of a Symposium held at the National Physical Laboratory, Nov. 1958\*, vol. 1, pp. 421-456. Londres: HM Stationery Office, 1959. F. Rosenblatt, "Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms", Washington, DC: Spartan Books, 1962. C. von der Malsburg, "Self-organizing of orientation sensitive cells in the striate cortex", Kybernetik\*, vol. 14, pp. 85-100, 1973. S. Grossberg, "Adaptive pattern classification and universal recoding: Parallel development and coding of neural feature detectors", Biolog. Cybernetics\*, vol. 23, pp. 121-134, 1976. K. Fukushima, "Cognitron: A self-organizing multilayered neural network", Biolog. Cybernetics\*, vol. 20, pp. 121-136, 1975. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position", Biolog. Cybernetics\*, vol. 36, pp. 193-202, 1980. B. Widrow, "Bootstrap learning in threshold logic systems", presentado en el American Automatic Control Council (Theory Committee), IFAC Meeting, Londres, Inglaterra, junio de 1966. B. Widrow, N. K. Gupta, y S. Maitra, "Punish/reward: Learning with a critic in adaptive threshold systems", IEEE Trans. Syst., Man, Cybernetics\*, vol. SMC-3, pp. 455-465, sept. 1973. A. G. Barto, R. S. Sutton, y C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems", IEEE Trans. Syst., Man, Cybernetics\*, vol. SMC-13, pp. 834-846, 1983. J. S. Albus, "A new approach to manipulator control: the

- [14] W. T. Miller, III, "Sensor-based control of robotic manipulators using a general learning algorithm", IEEE J. Robotics Automat., vol. RA-3, pp. 157-165, abr. 1987. [15] S. Grossberg, "Adaptive pattern classification and universal recoding", Biolog. Cybernetics\*, vol. 23, pp. 187-202, 1977. [16] G. A. Carpenter y S. Grossberg, "A massively parallel architecture for a self-organizing neural pattern recognition machine", Computer Vision, Graphics, and Image Processing\*, vol. 37, pp. 54-115, 1983. [17] "Art 2: Self-organization of stable category recognition codes for analog output patterns", Applied Optics\*, vol. 26, pp. 4919-4930, 1987. [18] "Art 3 hierarchical search: Chemical transmitters in self-organizing pattern recognition architectures", Proc. Int. Joint Conf. on Neural Networks\*, vol. 2, pp. 30-33, Wash., DC, ene. 1989. [19] T. Kohonen, "Self-organized formation of topologically correct feature maps", Biolog. Cybernetics\*, vol. 43, pp. 59-69, 1982. [20] "Self-Organization and Associative Memory", Nueva York: Springer-Verlag, 2.<sup>a</sup> ed., 1987. [21] D. O. Hebb, "The Organization of Behavior", Nueva York: Wiley, 1949. [22] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities", Proc. Natl. Acad. Sci., vol. 79, pp. 2554-2558, abr. 1982. [23] "Neurons with graded response have collective computational properties like those of two-state neurons", Proc. Natl. Acad. Sci., vol. 81, pp. 3088-3092, may. 1984. [24] B. Kosko, "Adaptive bidirectional associative memories", Appl. Optics\*, vol. 26, pp. 4947-4960, 1 dic. 1987. [25] G. E. Hinton, R. J. Sejnowski y D. H. Ackley, "Boltzmann machines: Constraint satisfaction networks that learn", Tech. Rep. CMU-CS-84-119, Carnegie-Mellon University, Dept. of Computer Science, 1984. [26] "Learning and relearning in Boltzmann machines", en "Parallel Distributed Processing", vol. 1, cap. 7, D. E. Rumelhart y J. L. McClelland, Eds. Cambridge, MA: M.I.T. Press, 1986. [27] L. R. Talbert et al., "A real-time adaptive speech-recognition system", Tech. rep., Stanford University, 1963. [28] M. J. C. Hu, "Application of the Adaline System to Weather Forecasting", Tesis, Tech. Rep. 6775-1, Stanford Electron. Labs., Stanford, CA, jun. 1964. [29] B. Widrow, "The original adaptive neural net broom-balance problem", IEEE Intl. Symp. Circuits and Systems\*, pp. 351-357, Phila., PA, 4-7 may. 1987. [30] B. Widrow y S. D. Stearns, "Adaptive Signal Processing", Englewood Cliffs, NJ: Prentice-Hall, 1985. [31] B. Widrow, P. Mantey, L. Griffiths y B. Goode, "Adaptive antenna systems", Proc. IEEE\*, vol. 55, pp. 2143-2159, dic. 1967. [32] B. Widrow, "Adaptive inverse control", Proc. 20th Intl. Fed. of Automatic Control Workshop\*, pp. 1-5, Lund, Suecia, jul. 1983. [33] B. Widrow et al., "Adaptive noise cancelling: Principles and applications", Proc. IEEE\*, vol. 63, pp. 1692-1716, dic. 1975. [34] R. W. Lucky, "Automatic equalization for digital communication", Bell Syst. Tech. J., vol. 44, pp. 547-588, abr. 1965. [35] R. W. Lucky et al., "Principles of Data Communication", Nueva York: McGraw-Hill, 1968. [36] M. M. Sondhi, "An adaptive echo canceller", Bell Syst. Tech. J., vol. 46, pp. 497-511, mar. 1967. [37] P. Werbos, "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences", Tesis doctoral, Harvard University, Cambridge, MA, ago. 1974. [38] Y. Le Cun, "A theoretical framework for back-propagation", en "Proc. 1988 Connectionist Models Summer School", D. Touretzky, G. Hinton y T. Sejnowski, Eds., 17-26 jun., pp. 21-28. San Mateo, CA: Morgan Kaufmann. [39] D. Parker, "Learning-logic", Invention Report S81-64, File 1, Office of Technology Licensing, Stanford University, Stanford, CA, oct. 1980. "Learning-logic", Technical Report TR-47, Center for

Aquí está la traducción al español académico formal, siguiendo todas las reglas específicas de la investigación Computacional en Economía y Ciencias de la Gestión, M. I.T., Abr. 1985. D. E. Rumelhart, G. E. Hinton, y R. J. Williams, "Learning internal representations by error propagation", ICS Report 8506, Institute for Cognitive Science, University of California at San Diego, La Jolla, CA, Sept. 1985.

[41] Learning internal representations by error propagation", en Parallel Distributed Processing, vol. 1, cap. 8, D. E. Rumelhart y J. L. McClelland, Eds., Cambridge, MA: M.I.T. Press, 1986.

[42] B. Widrow, R. G. Winter, y R. Baxter, "Learning phenomena in layered neural networks", Proc. 1st IEEE Intl. Conf. on Neural Networks, vol. 2, pp. 411-429, San Diego, CA, Jun. 1987.

[43] R. P. Lippmann, "An introduction to computing with neural nets", IEEE ASSP Mag., Abr. 1987.

[44] J. A. Anderson y E. Rosenfeld, Eds., Neurocomputing: Foundations of Research, Cambridge, MA: M.I.T. Press, 1988.

[45] N. Nilsson, Learning Machines. New York: McGraw-Hill, 1965.

[46] D. E. Rumelhart y J. L. McClelland, Eds., Parallel Distributed Processing, Cambridge, MA: M.I.T. Press, 1986.

[47] B. Moore, "ART 1 and pattern clustering", en Proc. 1988 Connectionist Models Summer School, D. Touretzky, G. Hinton, y T. Sejnowski, Eds., Jun. 17-26 1988, pp. 174-185, San Mateo, CA: Morgan Kaufmann.

[48] DARPA Neural Network Study, Fairfax, VA: AFCEA International Press, 1988.

[49] D. Nguyen y B. Widrow, "The truck backer-upper: An example of self-learning in neural networks", Proc. Intl. Joint Conf. on Neural Networks, vol. 2, pp. 357-363, Wash., DC, Jun. 1989.

[50] T. J. Sejnowski y C. R. Rosenberg, "Nettalk: a parallel network that learns to read aloud", Tech. Rep. JHU/EECS-86/01, Johns Hopkins University, 1986.

[51] Parallel networks that learn to pronounce English text", Complex Systems, vol. 1, pp. 145-168, 1987.

[52] P. M. Shea y V. Lin, "Detection of explosives in checked airline baggage using an artificial neural system", Proc. Intl. Joint Conf. on Neural Networks, vol. 2, pp. 31-34, Wash., DC, Jun. 1989.

[53] D. G. Bounds, P. J. Lloyd, B. Mathew, y G. Waddell, "A multilayer perceptron network for the diagnosis of low back pain", Proc. 2d IEEE Intl. Conf. on Neural Networks, vol. 2, pp. 481-489, San Diego, CA, Jul. 1988.

[54] G. Bradshaw, R. Fozzard, y L. Ceci, "A connectionist expert system that actually works", en Advances in Neural Information Processing Systems 1, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1989, pp. 248-255.

[55] N. Makhov, "Neural nets making the leap out of lab", Electronic Engineering Times, p. 1, Ene. 22, 1990.

[56] C. A. Mead, Analog VLSI and Neural Systems. Reading, MA: Addison-Wesley, 1989.

[57] B. Widrow y M. E. Hoff, Jr., "Adaptive switching circuits", 1960 IRE Western Electric Show and Convention Record, Part 4, pp. 96-104.

[58] Ago. 23, 1960.

[59] "Adaptive switching circuits", Tech. Rep. 1553-1, Stanford Electron. Labs., Stanford, CA, Jun. 30, 1960.

[60] P. M. Lewis II y C. Coates, Threshold Logic. New York: Wiley, 1967.

[61] T. M. Cover, Geometrical and Statistical Properties of Linear Threshold Devices. Ph.D. thesis, Tech. Rep. 6107-1, Stanford Electron. Labs., Stanford, CA, Jun. 1964.

[62] R. O. Winder, Threshold Logic. Ph.D. thesis, Princeton University, Princeton, NJ, 1962.

[63] S. H. Cameron, "An estimate of the complexity requisite in a universal decision network", Proc. 1960 Bionics Symposium, Wright Air Development Division Tech. Rep. 60-600, pp. 197-211, Dayton, OH, Dic. 1960.

[64] R. D. Joseph, "The number of orthants in n-space intersected by an s-dimensional subspace", Tech. Memorandum 8, Project PARA, Cornell Aeronautical Laboratory, Buffalo, New York, 1960.

[65] D. F. Specht, Generation of Polynomial Discriminant Functions.

[67] [68] [69] [70] [71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90] tions for Pattern Recognition. Ph.D. thesis, Tech. Rep. 6764-5, Stanford Electron. Labs., Stanford, CA, May 1968.

[91] Electrocardiographic diagnosis using the polynomial discriminant method of pattern recognition; IEEE Trans. Biomed. Eng., vol. BME-14, pp. 90-95, Apr. 1967.

[92] "Generation of polynomial discriminant functions for pattern recognition", IEEE Trans. Electron. Comput., vol. EC-16, pp. 308-319, June 1967.

[93] A. R. Barron, "Adaptive learning networks: Development and application in the United States of algorithms related to gradient descent", in Self-Organizing Methods in Modeling, S. J. Farlow, Ed., New York: Marcel Dekker Inc., 1984, pp. 25-65.

[94] "Predicted squared error: A criterion for automatic model selection", in Self-Organizing Methods in Modeling, in S. J. Farlow, Ed. New York: Marcel Dekker Inc., 1984, pp. 87-103.

[95] A. R. Barron and R. L. Barron, "Statistical learning networks: A unifying view", 1988 Symp. on the Interface: Statistics and Computing Science, pp. 192-203, Reston, VA, Apr. 21-23, 1988.

[96] A. G. Ivakhnenko, Polynomial theory of complex systems, IEEE Trans. Syst., Man, Cybernetics, SMC-1, pp. 364-378, Oct. 1971.

[97] Y. H. Pao, Functional link nets: Removing hidden layers, Appl. Artif. Intell. Expt., pp. 60-68, Apr. 1989.

[98] C. L. Giles and T. Maxwell, "Learning, invariance, and generalization in high-order neural networks", Appl. Optics, vol. 26, pp. 4972-4978, Dec. 1, 1987.

[99] M. E. Hoff, Jr., Learning Phenomena in Networks of Adaptive Switching Circuits. Ph.D. thesis, Tech. Rep. 1554-1, Stanford Electron. Labs., Stanford, CA, July 1962.

[100] W. C. Ridgway III, An Adaptive Logic System with Generalizing Properties. Ph.D. thesis, Tech. Rep. 1556-1, Stanford Electron. Labs., Stanford, CA, April 1962.

[101] F. H. Glanz, Statistical Extrapolation in Certain Adaptive Pattern-Recognition Systems. Ph.D. thesis, Tech. Rep. 6767-1, Stanford Electron. Labs., Stanford, CA, May 1965.

[102] B. Widrow, "Adaptive and Madaline 1963, plenary speech", Proc. 1st IEEE Intl. Conf. on Neural Networks, vol. 1, pp. 145-158, San Diego, CA, June 23, 1987.

[103] "An adaptive-adaline neuron using chemical elements", Tech. Rep. 1553-2, Stanford Electron. Labs., Stanford, CA, Oct. 17, 1960.

[104] C. L. Giles, R. D. Griffin, and T. Maxwell, "Encoding geometric invariances in higher order neural networks", Neural Information Processing Systems, in D. Z. Anderson, Ed. New York: American Institute of Physics, 1988, pp. 301-309.

[105] D. Casasent and D. Psaltis, "Rotation, rotation, and scale invariant optical correlation", Appl. Optics, vol. 15, pp. 1795-1799, July 1976.

[106] W. L. Reber and J. Lymn, "An artificial neural system design for rotation and scale invariant pattern recognition", Proc. 1st IEEE Intl. Conf. on Neural Networks, vol. 4, pp. 277-283, San Diego, CA, June 1987.

[107] B. Widrow and R. G. Winter, "Neural nets for adaptive filtering and adaptive pattern recognition", IEEE Computer, pp. 25-39, Mar. 1988.

[108] A. Khotanadzad and Y. H. Hong, "Rotation invariant pattern recognition using zernike moments", Proc. 9th Intl. Conf. on Pattern Recognition, vol. 1, pp. 326-328, 1988.

[109] C. von der Malsburg, "Pattern recognition by labeled graph matching", Neural Networks, vol. 1, pp. 141-148, 1988.

[110] A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, and K. J. Lang, "Phoneme recognition using time delay neural networks", IEEE Trans. Acoust., Speech, and Signal Processing, vol. ASSP-37, pp. 328-339, Mar. 1989.

[111] C. M. Newman, "Memory capacity in neural network models: Rigorous lower bounds", Neural Networks, vol. 1, pp. 223-238, 1988.

[112] Y. S. Abu-Mostafa and J. St. Jacques, "Information capacity of the hopfield model", IEEE Trans. Inform. Theory, vol. IT-31, pp. 461-464, 1985.

[113] Y. S. Abu-Mostafa, "Neural networks for computing? in Neural Networks for Computing, Amer. Inst. of Phys. Conf. Proc. No. 151, J. S. Denker, Ed. New York: American Institute of Physics, 1986, pp. 1-6.

[114] S. S. Venkatesh, "Epsilon capacity of neural networks".

Aquí está la traducción al español académico formal, siguiendo todas las reglas especificadas:

Redes Neuronales para Computación, Amer. Inst. of Phys. Conf. Proc. No. 151, J. S. Denker, Ed. New York: American Institute of Physics, 1986, pp. 440-445. J. D. Greenfield, Practical Digital Design Using 20 ed., New York: Wiley, 1983. M. Stinchcombe y H. White, Universal approximation using feedforward networks with non-sigmoid hidden layer activation functions; Proc. Intl. Joint Conf. on Neural Networks, vol. 1, pp. 613-617, Wash., DC, June 1989. G. Cybenko, Continuous valued neural networks with two hidden layers are sufficient; Tech. Rep., Dept. of Computer Science, Tufts University, Mar. 1988. B. Irie y S. Miyake, Capabilities of three-layered perceptrons; Proc. 2d IEEE Intl. Conf. on Neural Networks, vol. 1, pp. 641-647, San Diego, CA, July 1988. M. L. Minsky y S. A. Papert, Perceptrons: An Introduction to Computational Geometry. Cambridge, MA: M.I.T. Press, expanded ed., 1988. M. W. Roth; Survey of neural network technology for automatic target recognition; IEEE Trans. Neural Networks, vol. 1, pp. 28-43, Mar. 1990. T. M. Cover, Capacity problems for linear machines; Pattern Recognition, in L. N. Kanal, Ed. Wash., DC: Thompson Book Co., 1968, pp. 283-289, part 3. E. B. Baum, On the capabilities of multilayer perceptrons; Complexity, vol. 4, pp. 193-215, Sept. 1988. A. Lapides y R. Farber, Low neural networks work; Tech. Rep. LA-UR-88-418, Los Alamos Nat. Laboratory, Los Alamos, NM, 1987. D. Nguyen y B. Widrow, Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights; Proc. Intl. Joint Conf. on Neural Networks, San Diego, CA, June 1990. G. Cybenko, Approximation by superpositions of a sigmoidal function; Mathematics of Control, Signals, and Systems, vol. 2, 1989. E. B. Baum y D. Hauskrecht, What size net gives valid generalization; Neural Computation, vol. 1, pp. 151-160, 1989. J. J. Hopfield y D. W. Tank, Neural computations of decisions in optimization problems; Biolog. Cybernetics, vol. 52, pp. 141-152, 1985. K. S. Narendra y K. Parthasarathy, Identification and control of dynamical systems using neural networks; IEEE Trans. Neural Networks, vol. 1, pp. 4-27, Mar. 1990. C. H. Mays, Adaptive Threshold Logic. Ph.D. thesis, Tech. Rep. 1557-1, Stanford Electron. Labs., Stanford, CA, Apr. 1963. F. Rosenblatt, On the convergence of reinforcement procedures in simple perceptrons; Cornell Aeronautical Laboratory Report VG-1196-G-4, Buffalo, NY, Feb. 1960. H. Block, The perceptron: A model for brain functioning, 1; Rev. Modern Phys., vol. 34, pp. 123-135, Jan. 1962. R. G. Winter, Madaline Rule II: A New Method for Training Networks of Adalines. Ph.D. thesis, Stanford University, Stanford, CA, Jan. 1989. E. Walach y B. Widrow, The least mean fourth (LMF) adaptive algorithm and its family; IEEE Trans. Inform. Theory, vol. IT-30, pp. 275-283, Mar. 1984. E. B. Baum y F. Wilczek, Supervised learning of probability distributions by neural networks; in Neural Information Processing Systems, D. Z. Anderson, Ed. New York: American Institute of Physics, 1988, pp. 52-61. S. A. Solla, E. Levin, y M. Fleisher, Accelerated learning in layered neural networks; Complex Systems, vol. 2, pp. 625-640, 1988. D. B. Park, Minimal algorithms for adaptive neural networks: Second order back propagation, second order direct propagation, and second order Hebbian learning; Proc. 1st IEEE Intl. Conf. on Neural Networks, vol. 2, pp. 593-600, San Diego, CA, June 1987. A. J. Owens y D. L. Filkin, Efficient training of the back propagation network by solving a system of stiff ordinary differential equations; Proc. Intl. Joint Conf. on Neural Networks, vol. 2, pp. 381-386, Wash., DC, June 1989. D. G. Luenberger, Linear and Nonlinear Programming. Reading, MA: Addison-Wesley, 2d ed., 1984. A. Kramer y A. Sangiovanni-Vincentelli, Efficient parallel learning algorithms for neural networks; in Advances in

[116] [117] [118] [119] [120] [121] [122] [123] [124] [125] [126] [127] [128] [129] [130] [131] [132] [133] Neural Information Processing Systems 3, S. Touretzky, Ed., pp. 40-48, San Mateo, CA: Morgan Kaufmann, 1989. R. V. Sutherland, Relaxation Methods in Engineering Science. New York: Oxford, 1940. D. J. Wilde; Optimum Seeking Methods. Englewood Cliffs, NJ: Prentice-Hall, 1964. N. Wiener, Extrapolation, interpolation, and Smoothing of Stationary Time Series, with Engineering Applications. New York: Wiley, 1949. T. Kailath, A view of three decades of linear filtering theory; IEEE Trans. Inform. Theory, vol. 11-20, pp. 145-181, Mar. 1974. H. Bode y C. Shannon, A simplified derivation of linear least squares smoothing and prediction theory; IRE Trans. IRE, vol. 38, pp. 417-425, Apr. 1950. L. L. Horowitz y K. D. Sen, Performance advantage of complex LMS for controlling narrow-band adaptive arrays; IEEE Trans. Circuits, Systems, vol. CAS-28, pp. 562-576, June 1981.

Backpropagation rates within a joint conference on neural networks, vol. 1, pp. 639-642, Wash., DC, June 1989. . . . Backpropagation can give rise to spurious local minima even for networks without hidden layers; Complex Systems, vol. 3, pp. 91-106, 1989. J. J. Shynk y S. Roy, The LMS algorithm with momentum updating; SCAS 88, Espoo, Finland, June 1988. S. E. Fahmar, Faster learning variations on backpropagation: An empirical study; in Proc. 1988 Connectionist Models Summer School, D. Touretzky, G. Hinton, y T. Sejnowski, Eds. June 17-26, 1988, pp. 38-51, San Mateo, CA: Morgan Kaufmann. R. A. Jacobs, Increased rates of convergence through learning rate adaptation; Neural Networks, vol. 1, pp. 295-303, 1988. F. J. Pineda, Generalization of backpropagation to recurrent neural networks; Phys. Rev. Lett., vol. 18, pp. 2229-2232, 1987. L. B. Almeida, A learning rule for a synchronous perceptrons with feedback in a combinatorial environment; Proc. 1st IEEE Intl. Conf. on Neural Networks, vol. 2, pp. 609-618, San Diego, CA, June 1987. R. J. Williams y D. Zipser, A learning algorithm for continually running fully recurrent neural networks; CS Report 8805, Inst. for Cog. Sci., University of California at San Diego, La Jolla, CA, Oct. 1988. S. A. White, An adaptive recursive digital filter; Proc. 9th Asilomar Conf. Circuits Syst. Comput., p. 21, Nov. 1975. B. P. L. Moore, Learning state space trajectories in recurrent neural networks; Proc. 1988 Connectionist Models Summer School, D. Touretzky, G. Hinton, y T. Sejnowski, Eds. June 17-26, 1988, pp. 113-117. San Mateo, CA: Morgan Kaufmann. M. Holler, et al., An electrically trainable artificial neural network (ETANN) with 10240 floating gate synapses; Proc. Intl. Joint Conf. on Neural Networks, vol. 2, pp. 191-196, Wash., DC, June 1989. D. Andes, B. Widrow, M. Lehr, y E. Wan, MRILL: A robust algorithm for training analog neural networks; Proc. Intl. Joint Conf. on Neural Networks, vol. 1, pp. 533-536, Wash., DC, Jan. 1990.

Bernard Widrow (Fellow, IEEE) recibió los títulos de S. B., S.M. y Sc.D. del Instituto Tecnológico de Massachusetts en 1951, 1953 y 1956, respectivamente.

Él estuvo en el M.I.T. hasta que se incorporó al cuerpo docente de la Universidad de Stanford en 1959, donde actualmente es Profesor de ingeniería eléctrica. En la actualidad, se dedica a la investigación y la docencia en redes neuronales, reconocimiento de patrones, filtrado adaptativo y sistemas de control adaptativo. Es editor asociado

editor de las revistas \*Adaptive Control and Signal Processing\*, \*Neural Networks\*, \*Information Sciences\* y \*Pattern Recognition\*, y coautor junto con S. D. Stearns de \*Adaptive Signal Processing\* (Prentice Hall).

El Dr. Widrow recibió los títulos de SB, SM y ScD del MIT en 1951, 1953 y 1956, respectivamente. Es miembro de la Asociación Estadounidense de Profesores Universitarios, la Sociedad de Reconocimiento de Patrones, Sigma Xi y Tau Beta Pi. Es fellow de la Asociación Estadounidense para el Avance de la Ciencia. Es presidente de la Sociedad Internacional de Redes Neuronales. Recibió la Medalla Alexander Graham Bell del IEEE en 1986 por sus contribuciones excepcionales al avance de las telecomunicaciones.

Michael A. Lehr nació en Nueva Jersey el 18 de abril de 1964. Obtuvo el título de B.E.E. en ingeniería eléctrica en el Instituto de Tecnología de Georgia en 1987, graduándose como el primero de su clase. Recibió el M.S.E.E. de la Universidad de Stanford en 1986.

De 1982 a 1984, trabajó en el desarrollo de radios bidireccionales en Motorola en Ft. Lauderdale, Florida, y de 1984 a 1987 participó en el desarrollo y prueba de sistemas de sonar naval en el RBM en Manassas.

Virginia. Actualmente, es candidato a doctor en el Departamento de Ingeniería Eléctrica de la Universidad de Stanford. Sus intereses de investigación incluyen procesamiento adaptativo de señales y redes neuronales.

El Sr. Lehr posee una Licencia de Operador General de Radiotelefonía (1981) y una Habilitación de Radar (1982), y es miembro de Tau Beta Pi, Eta Kappa Nu y Phi Kappa Phi.